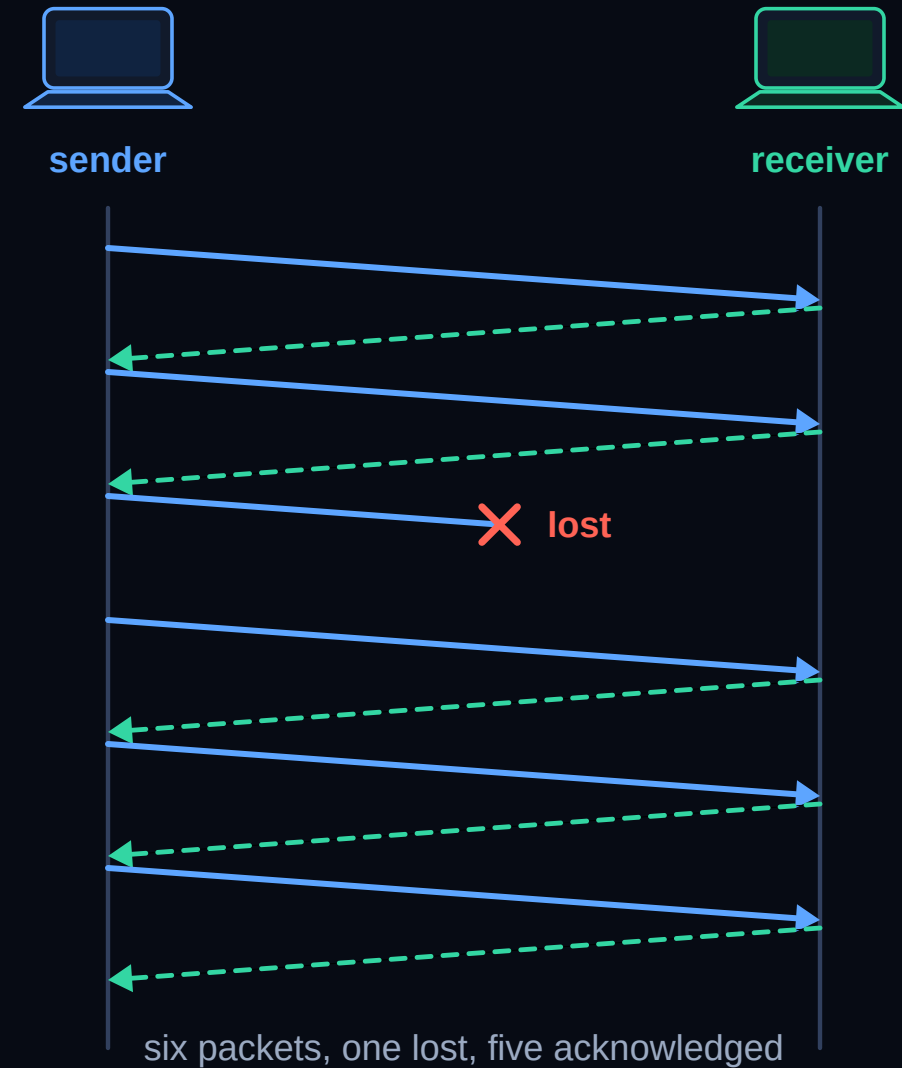


Reliable Data Transfer

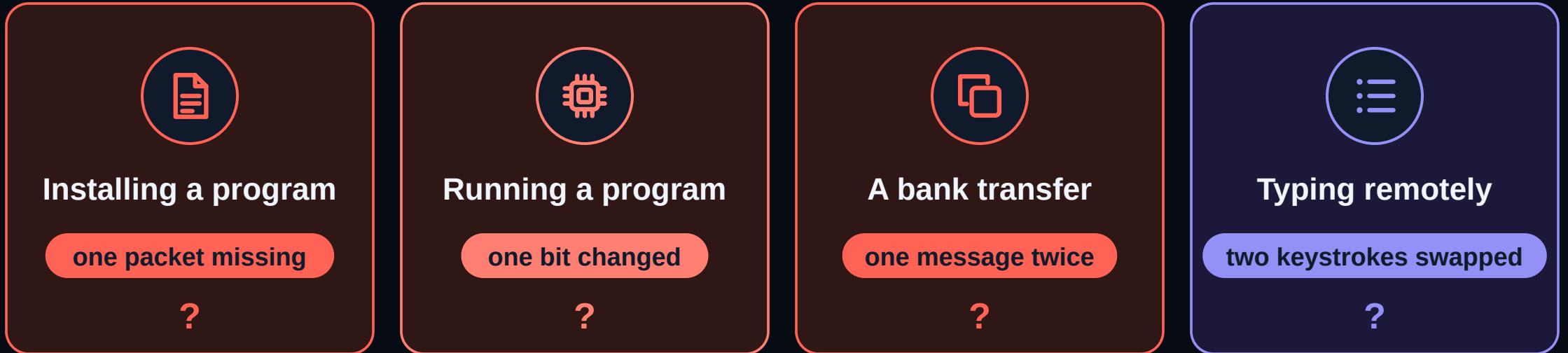
Building a service that does not lose data

The channel below delivers a packet if it can. The application above wants every byte, once, in order. This lecture builds the layer in between, one mechanism at a time, and measures what each mechanism costs.

Yaxiong Xie

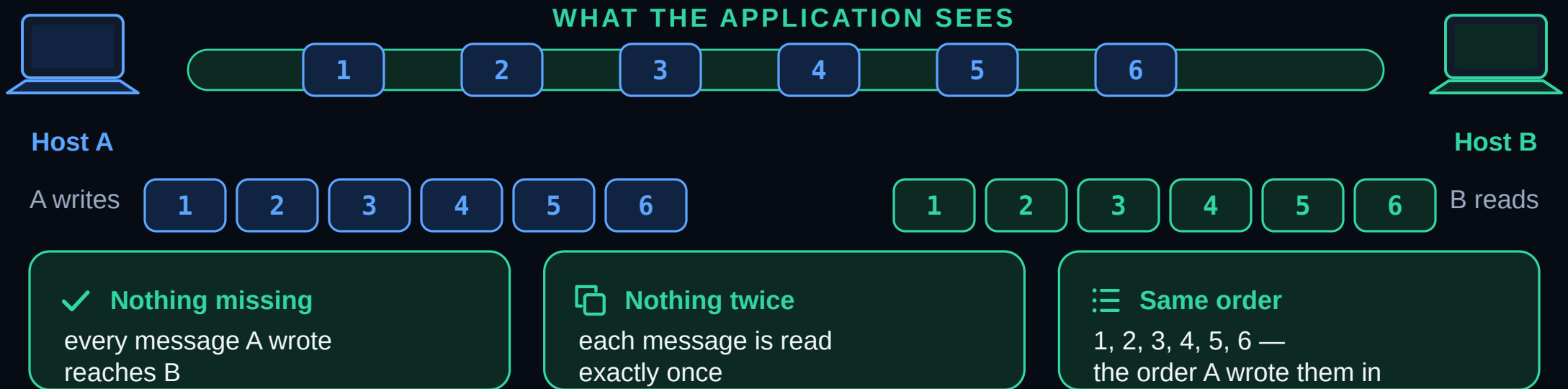


Why does the application need every byte, once, in order?



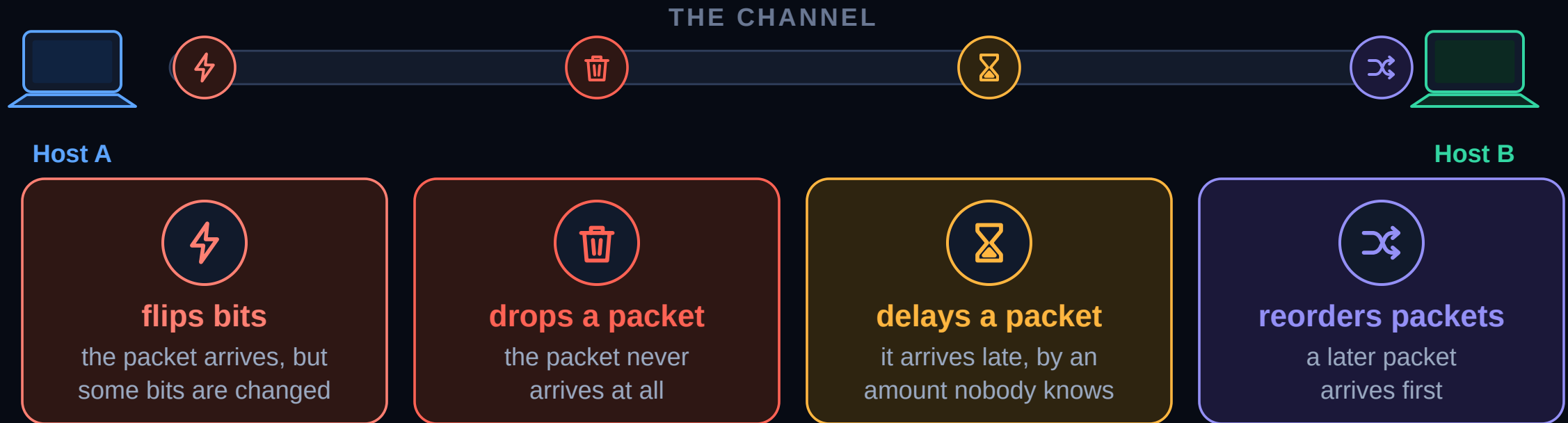
One small change on the way, and the application gets a wrong result.

The application wants every message, once, in order



This is the service the application is written against. It does not see the channel.

Does the channel below deliver this service?

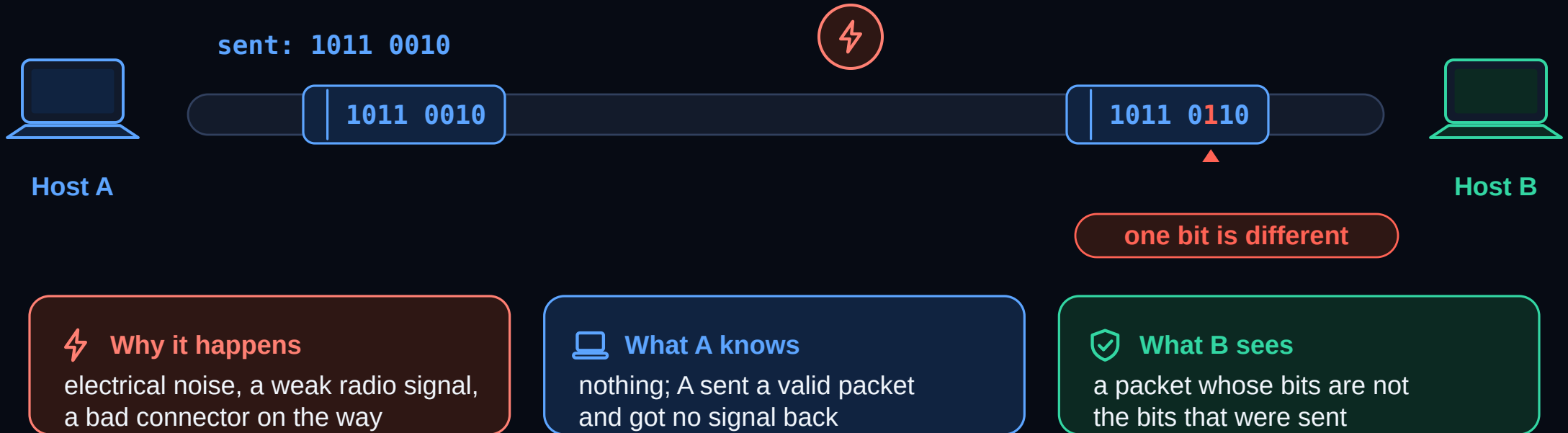


It may also deliver one packet twice. That copy is made by a retransmission, later in this lecture.

No. It can fail in four ways and promises nothing about any single packet.

• The problem

A bit flip: the packet arrives with different bits



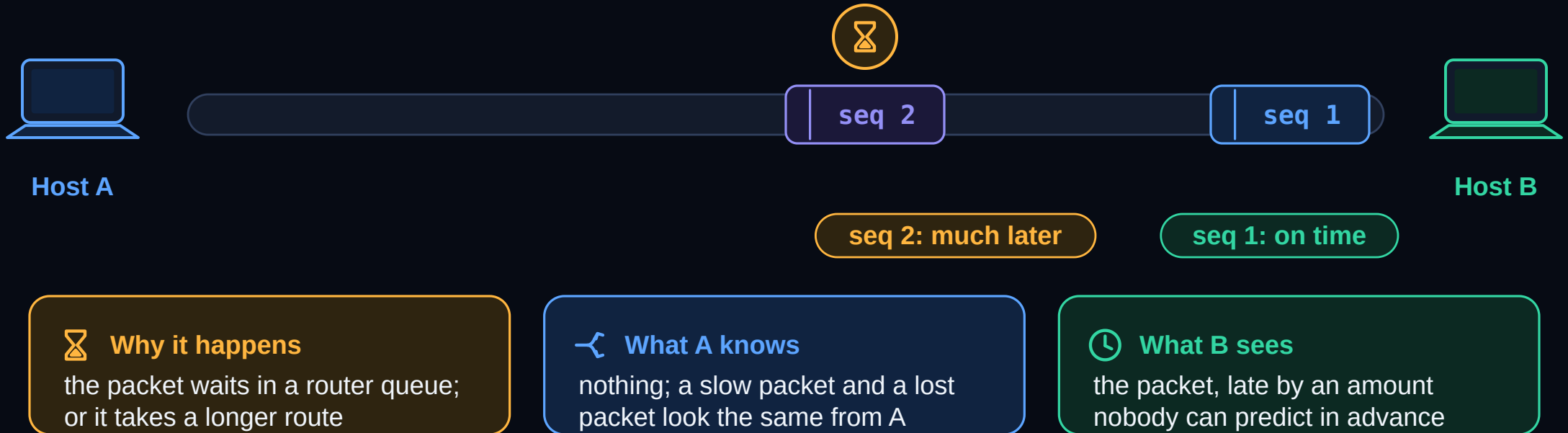
The bytes B delivers are not the bytes A sent, and nothing in the packet says so.

Loss: a router queue is full and the packet is discarded



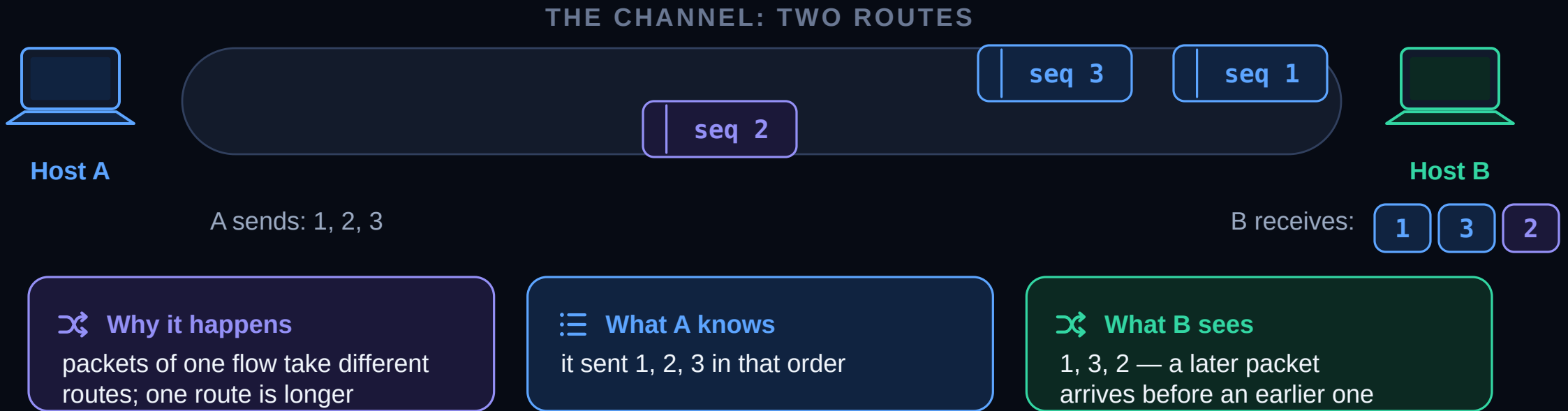
Neither end is told. Silence at B looks the same as A sending nothing.

Delay: a packet can arrive late by any amount



There is no upper bound. "Not here yet" never proves "lost".

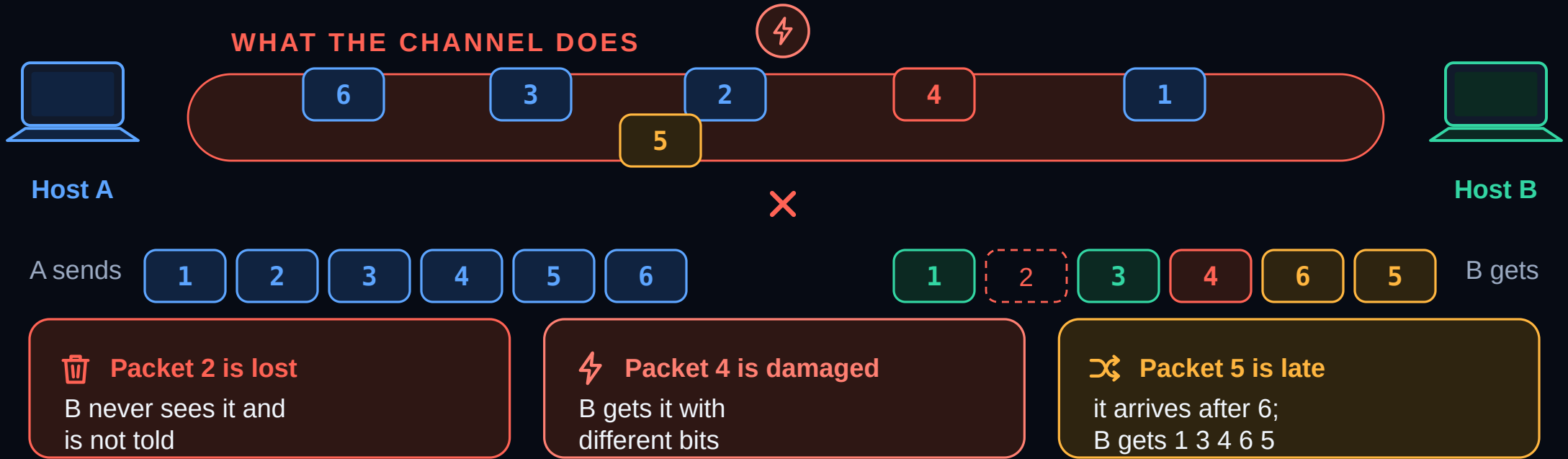
Reordering: a later packet can arrive first



Arrival order is not sending order. B needs another way to know the order.

• The problem

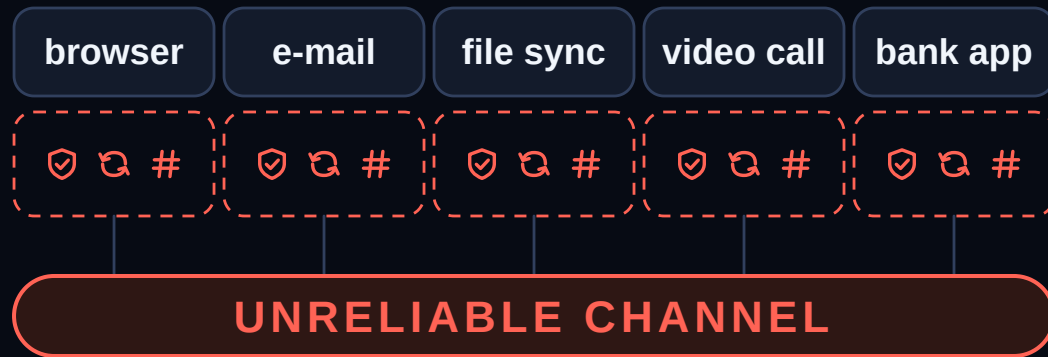
Below our layer: the same six packets, after the channel



Six packets in, five out: one missing, one wrong, two swapped. This is what B is given.

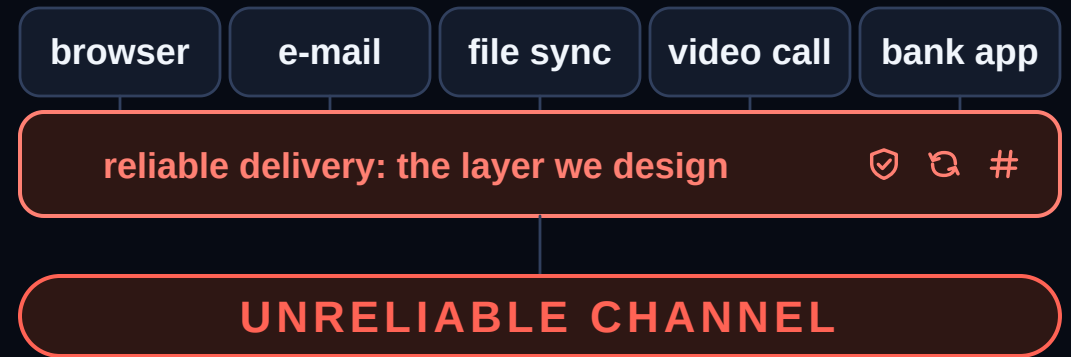
Should every application solve this itself?

INSIDE EVERY APPLICATION



five copies of the same code,
five places for the same bugs

ONCE, IN ONE LAYER BELOW THEM ALL



written once, tested once,
used by every application

The three icons are the tools this lecture builds: a check on each packet, a way to resend, a number on each packet.

Solve it once, in one layer that every application uses. That layer is this lecture.

This lecture: a reliable service on an unreliable channel

RELIABLE

what the application is promised

every byte

once

in order

correct

THE LAYER WE BUILD

this lecture: turn the channel below
into the service above

one problem at a time



bit errors
?



loss
?



delay
?



reordering
?

UNRELIABLE

what the channel actually does

⚡ flips bits

🗑 drops

⌚ delays

↔ reorders

Four defects below, four promises above: we build the layer between them.

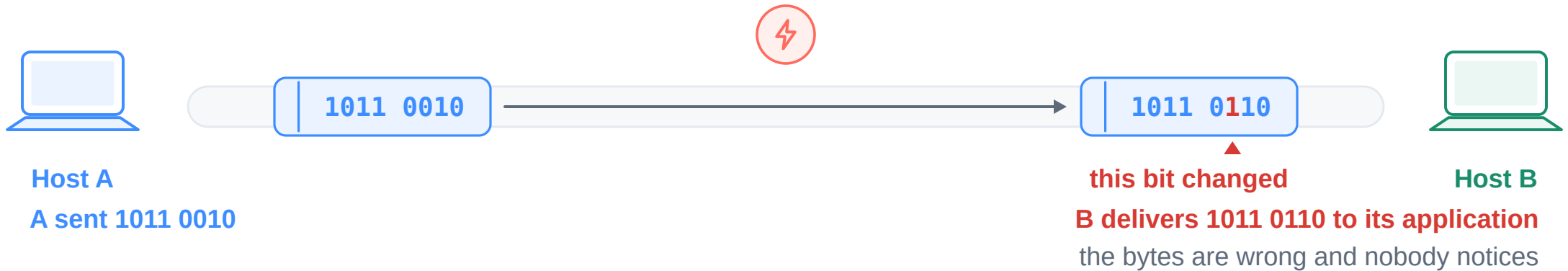
What if the channel only flips bits?

01

Bit errors

No loss and no reordering yet. Every packet arrives, and some arrive damaged. One step at a time: detect the damage, report it, repeat the packet, then fix what the report itself can break.

What happens when a bit flips on the way?



Before B trusts a packet, what has to happen?

A packet is only bits. Any pattern of bits is a valid packet.

B delivers wrong data. First job: B must be able to tell a damaged packet from a good one.

1 Bit errors

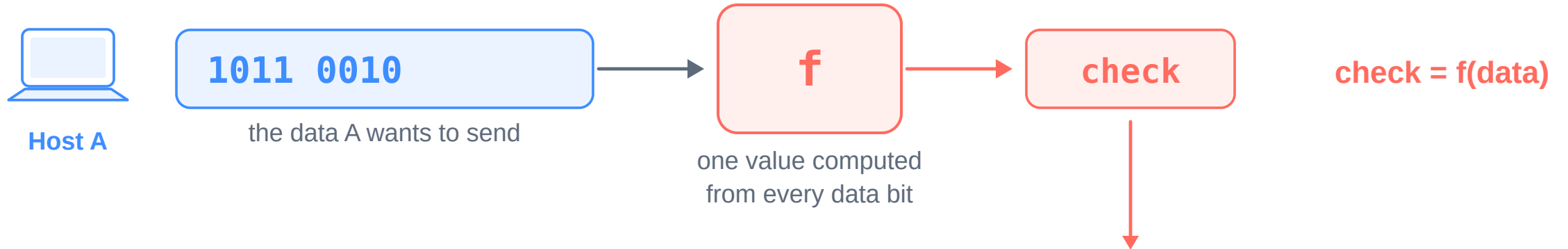
2 Loss

3 Delay

4 Reordering

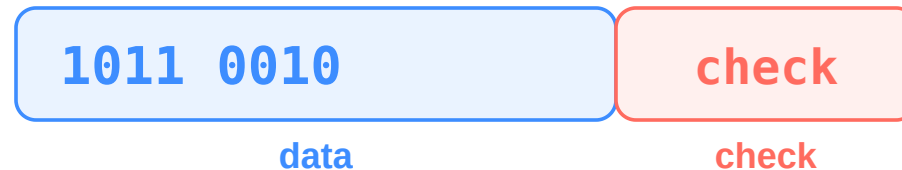
How can B tell that a packet is damaged?

AT A, BEFORE THE PACKET IS SENT



THE PACKET A SENDS

A computes the check once, and sends it with the data.



🕒 What f is exactly — a checksum, a CRC — comes later in the course. Here: any function of all the data bits.

A computes $check = f(data)$ and sends it in the packet, beside the data.

1 Bit errors

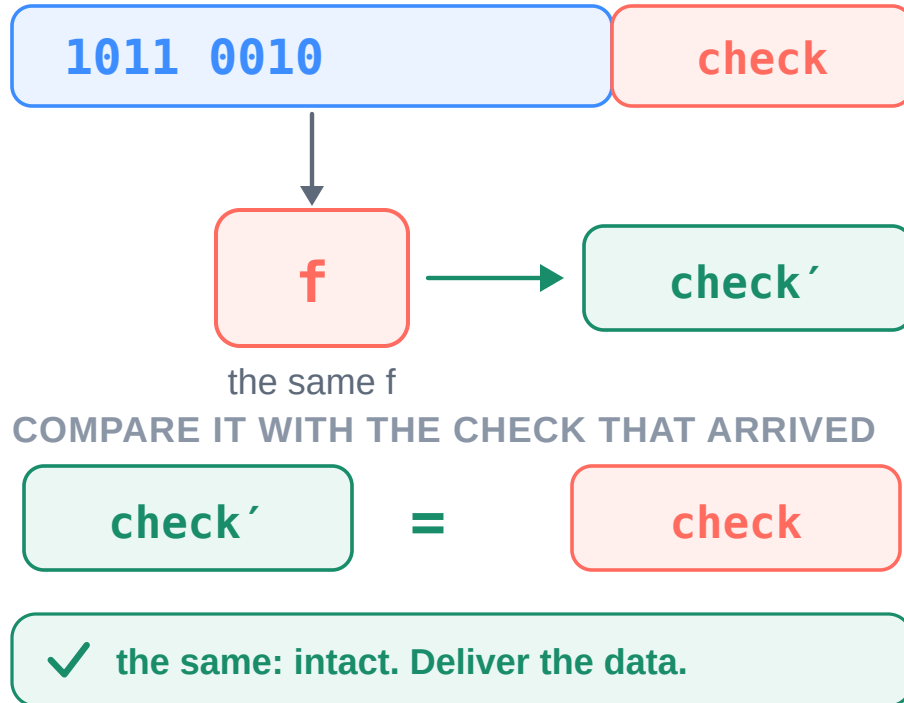
2 Loss

3 Delay

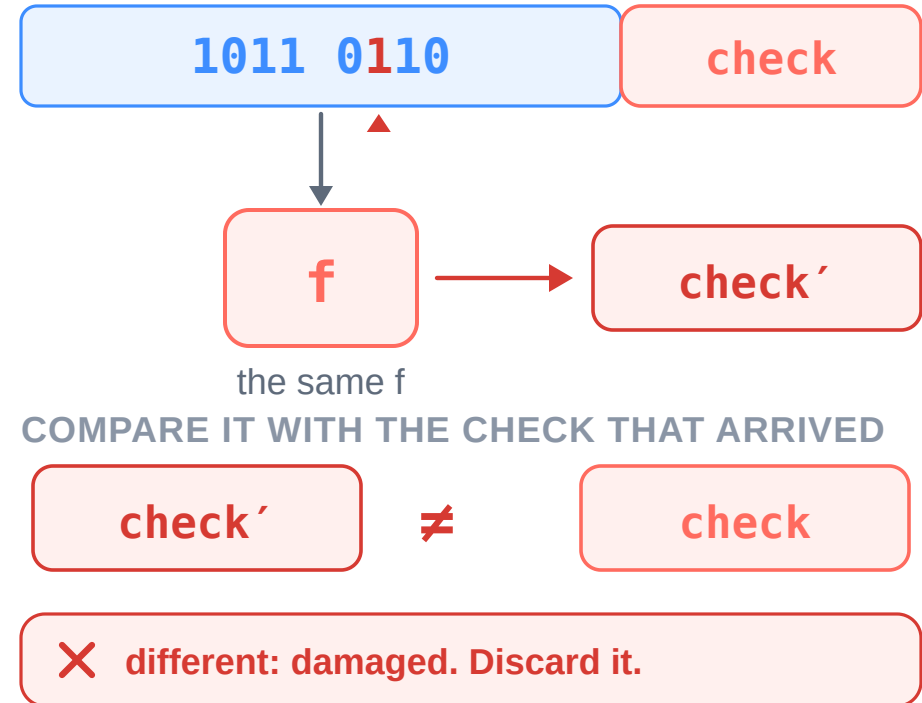
4 Reordering

What does B do with the check field?

THE PACKET ARRIVES INTACT



ONE BIT FLIPPED ON THE WAY



B computes f on the data that arrived and compares. Same: intact. Different: damaged.

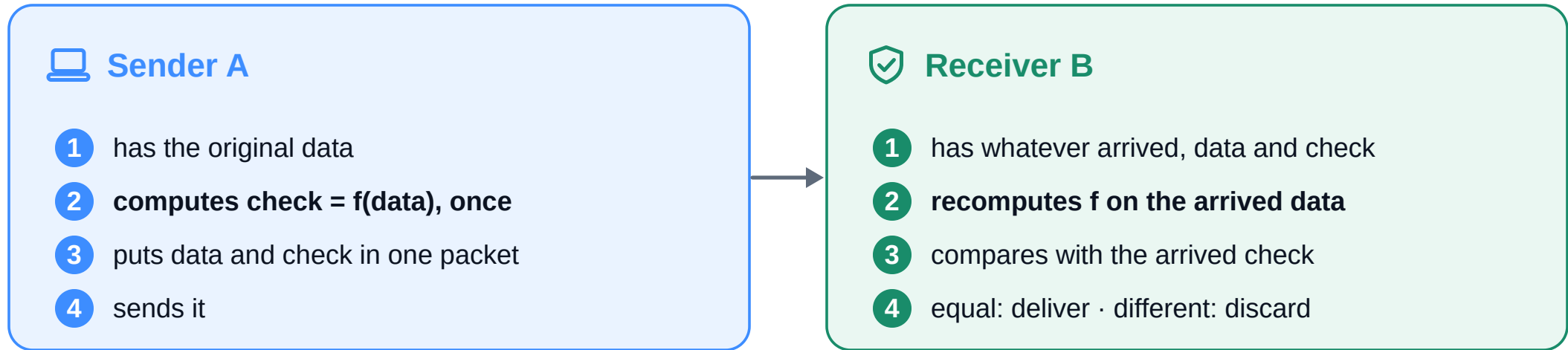
1 Bit errors

2 Loss

3 Delay

4 Reordering

Sender and receiver: who does what with the check



✓ **The same f at both ends. A computes it; B only verifies it.**

⚡ A flip inside the check field is caught the same way.

🕒 Two flips can cancel: no check catches every pattern.

B now knows good from damaged. A still knows nothing about what arrived.

1 Bit errors

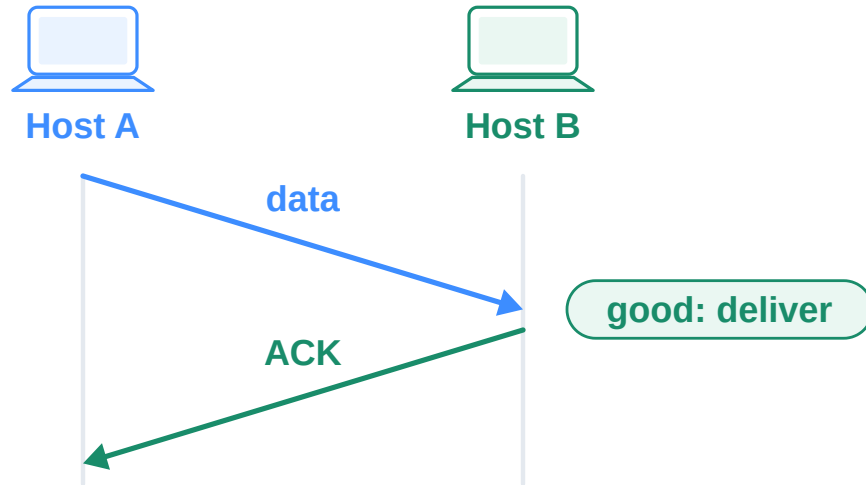
2 Loss

3 Delay

4 Reordering

How does A learn whether the packet arrived intact?

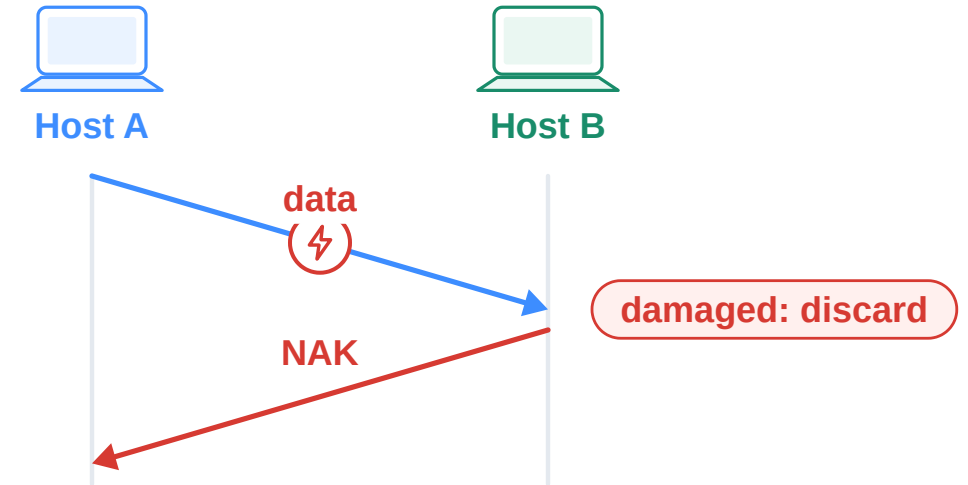
THE PACKET ARRIVED INTACT



ACK — "this packet was good"

B delivered it; A can move on

THE PACKET ARRIVED DAMAGED



NAK — "this packet was damaged"

B threw it away; A has to do something

B sends one small packet back: ACK or NAK. One bit of information.

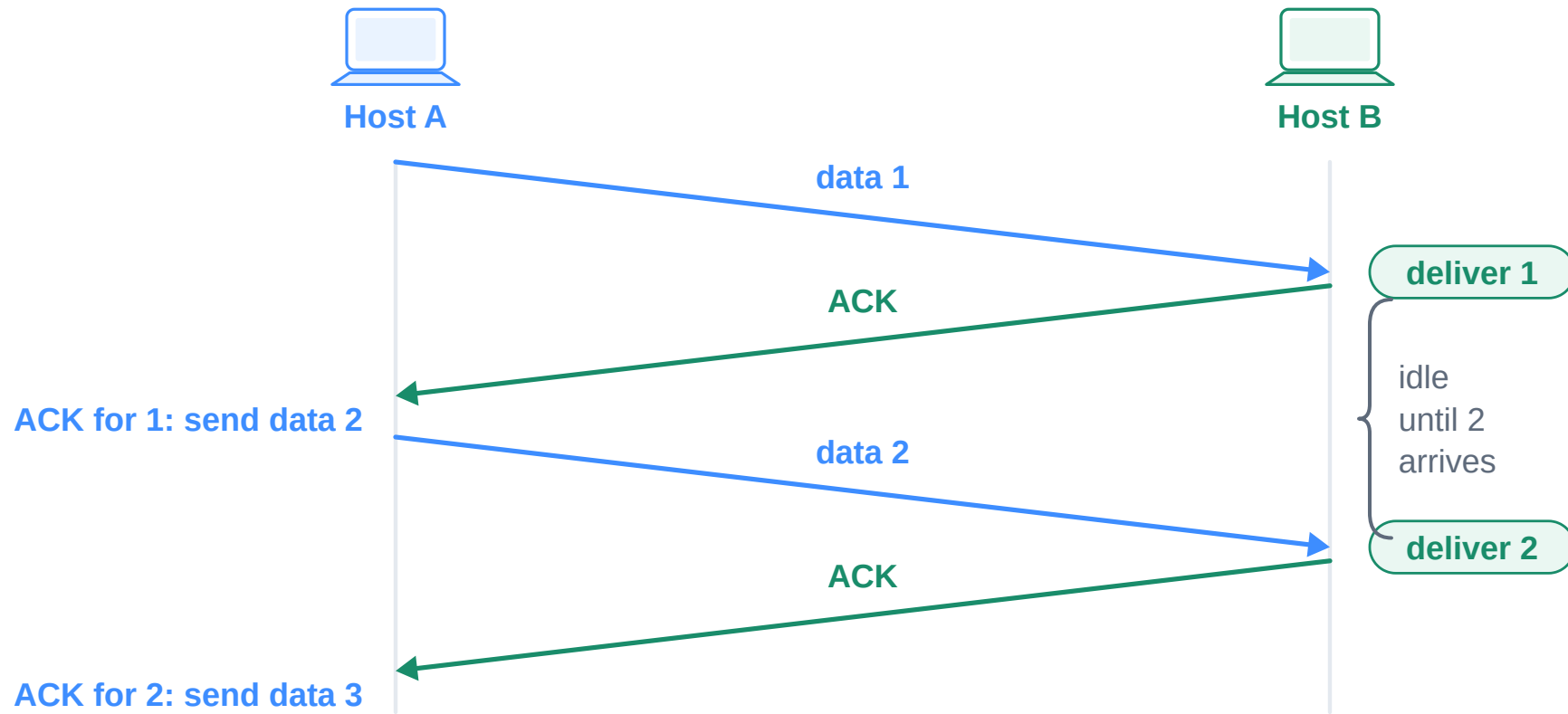
1 Bit errors

2 Loss

3 Delay

4 Reordering

What does A do after an ACK?



A sends the next packet. One packet at a time, each waiting for its verdict.

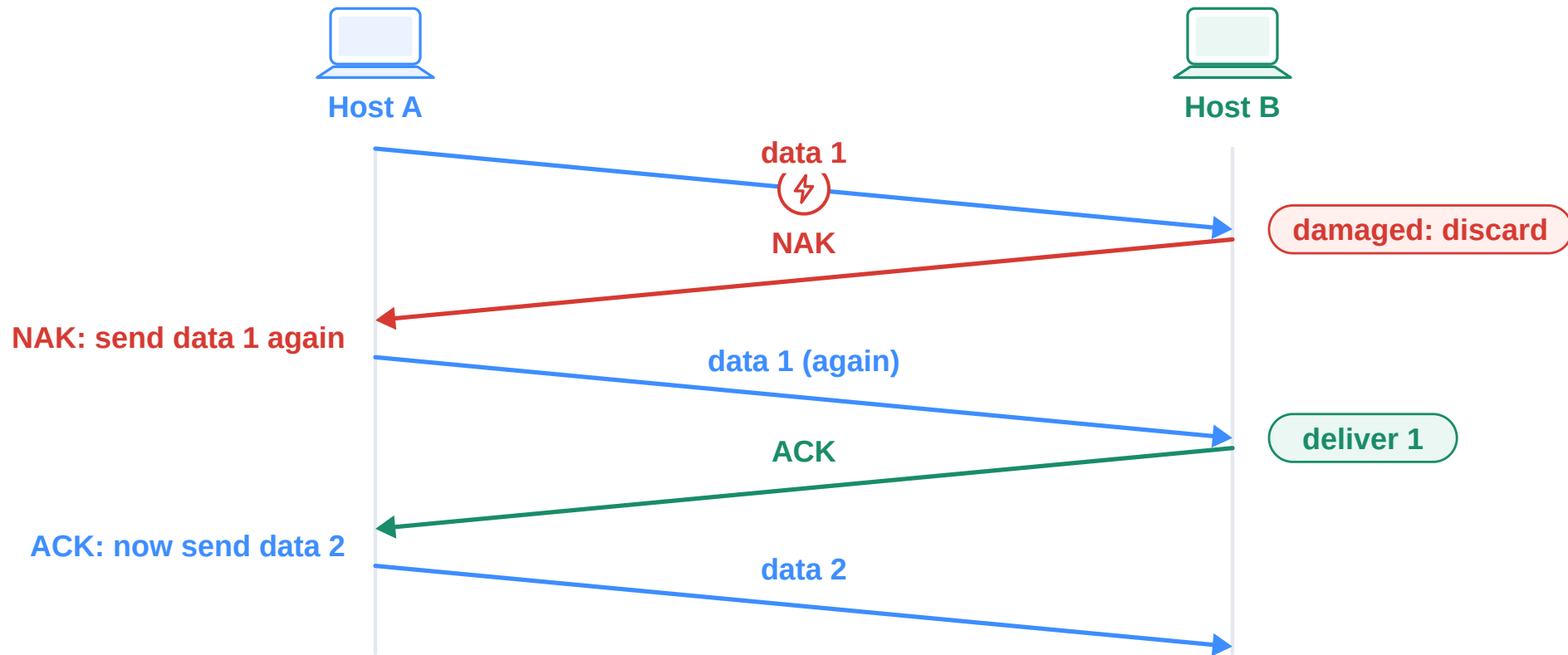
1 Bit errors

2 Loss

3 Delay

4 Reordering

What does A do after a NAK?



A sends the same packet again, and again, until B accepts it.

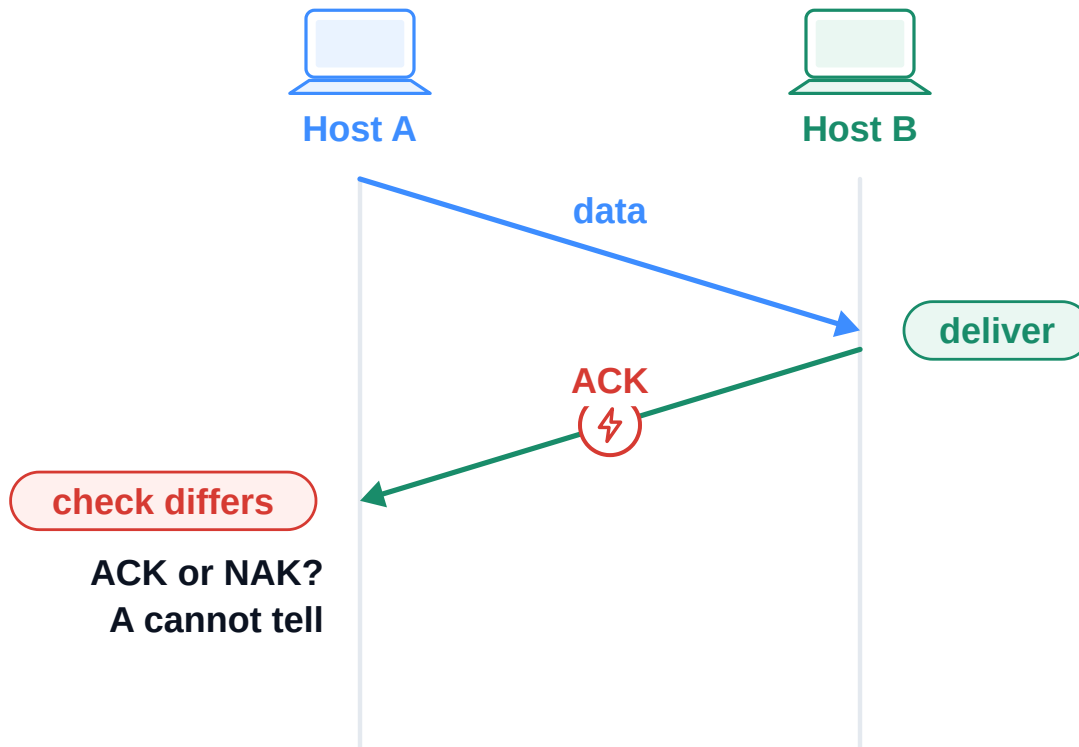
1 Bit errors

2 Loss

3 Delay

4 Reordering

What if the ACK or NAK itself is damaged?



WHAT A HOLDS



Feedback travels the same channel, so A checks it too. The check fails: A knows only that something arrived.

OPTION 1: ASSUME AN ACK, SEND THE NEXT

✗ If it was a NAK, B never gets this packet.
Data lost for good. Not safe.

OPTION 2: SEND THE SAME PACKET AGAIN

✓ If it was an ACK, B gets a copy.
Nothing is lost. Safe, but a new problem.

A cannot read the verdict. Its only safe option: send the packet again.

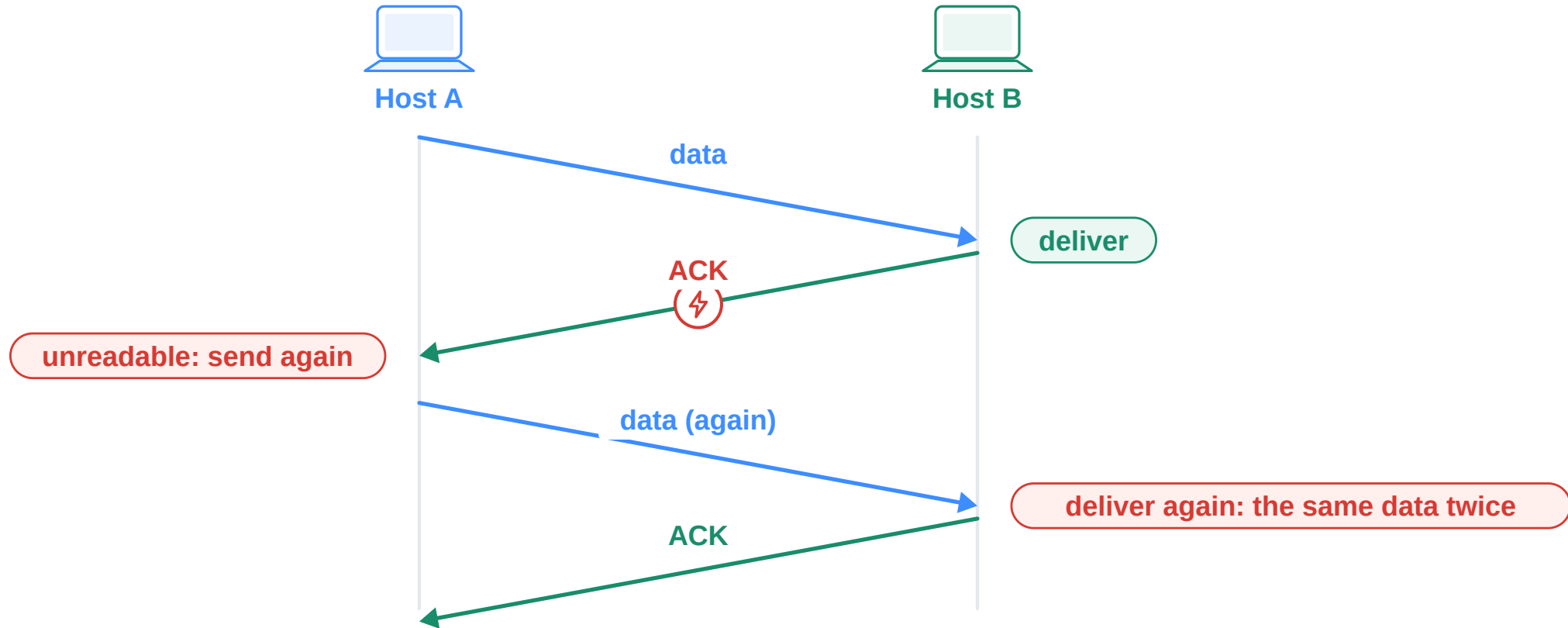
1 Bit errors

2 Loss

3 Delay

4 Reordering

Resending after damaged feedback gives B a duplicate



B delivers the same data twice, and nothing tells B that it did.

1 Bit errors

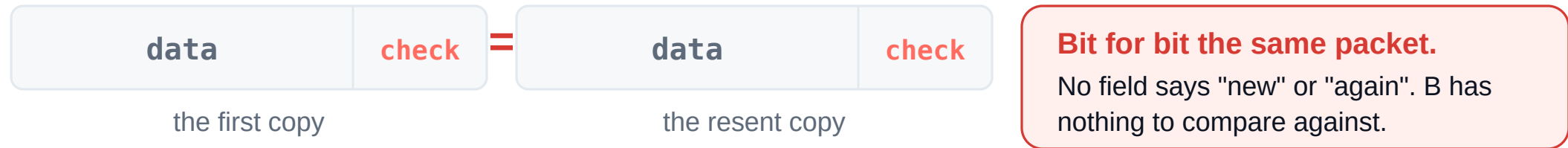
2 Loss

3 Delay

4 Reordering

How does B know that a packet is a repeat?

WHAT B RECEIVES, ONE AFTER THE OTHER



THE FIX: A FIELD THAT CHANGES WITH EVERY NEW PACKET



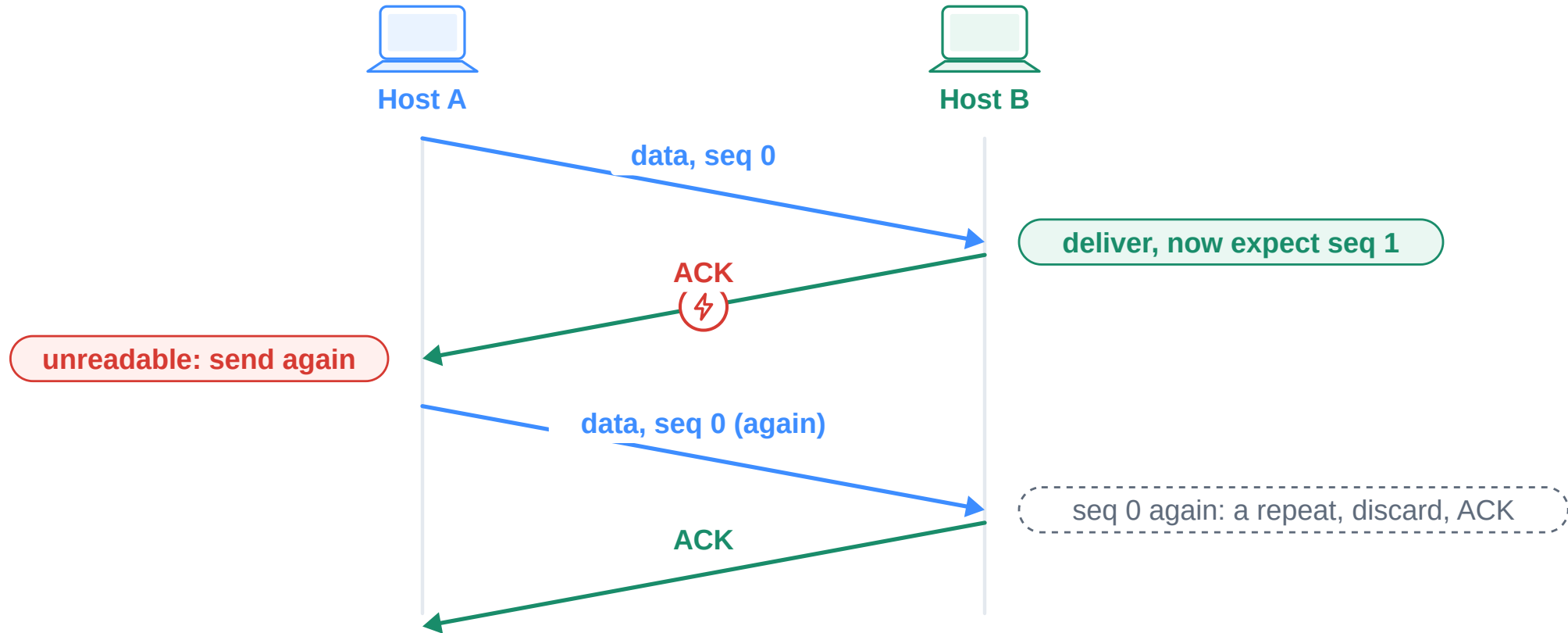
Each new packet gets a new number: 0, 1, 2, ... A retransmission carries the same number as the original.

B remembers the number of the last packet it delivered. The same number again means: a repeat, do not deliver.

Number the packets. A sequence number tells a repeat from new data.



With a sequence number, B recognizes the repeat



B discards the copy but still ACKs it, so A can move on to seq 1.

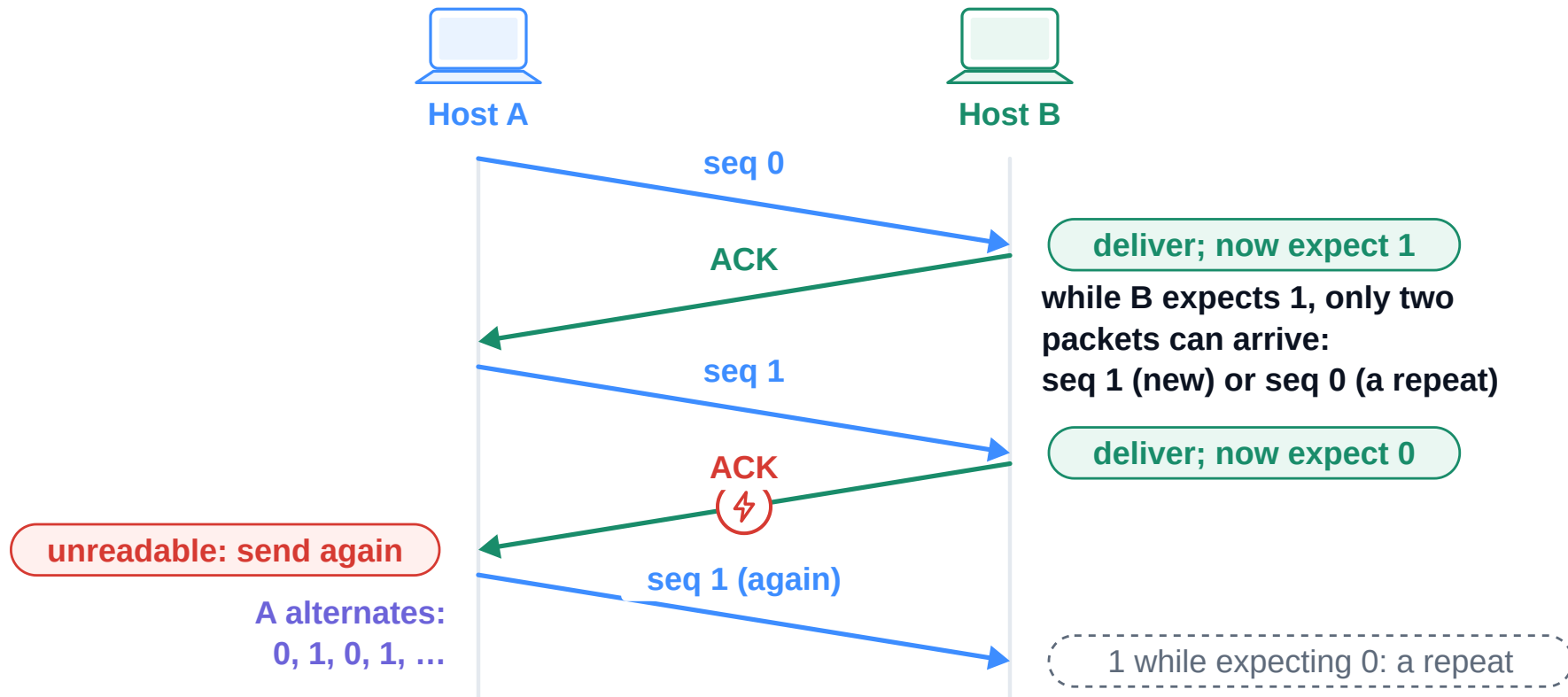
1 Bit errors

2 Loss

3 Delay

4 Reordering

How many sequence numbers are needed?



One bit. With one packet in flight, B only needs "the one I expect" or "the last one".

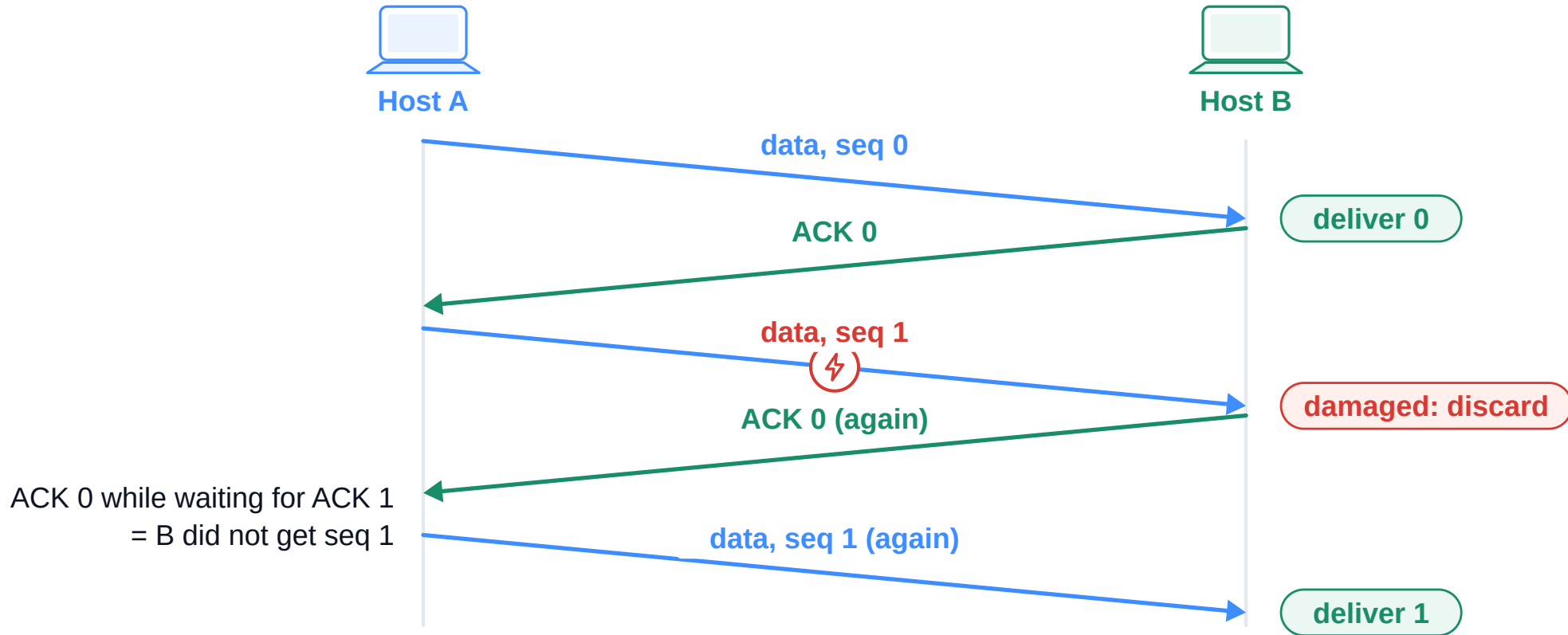
1 Bit errors

2 Loss

3 Delay

4 Reordering

Is the NAK needed at all?



No. B repeats the last ACK instead; the wrong number means: send again.

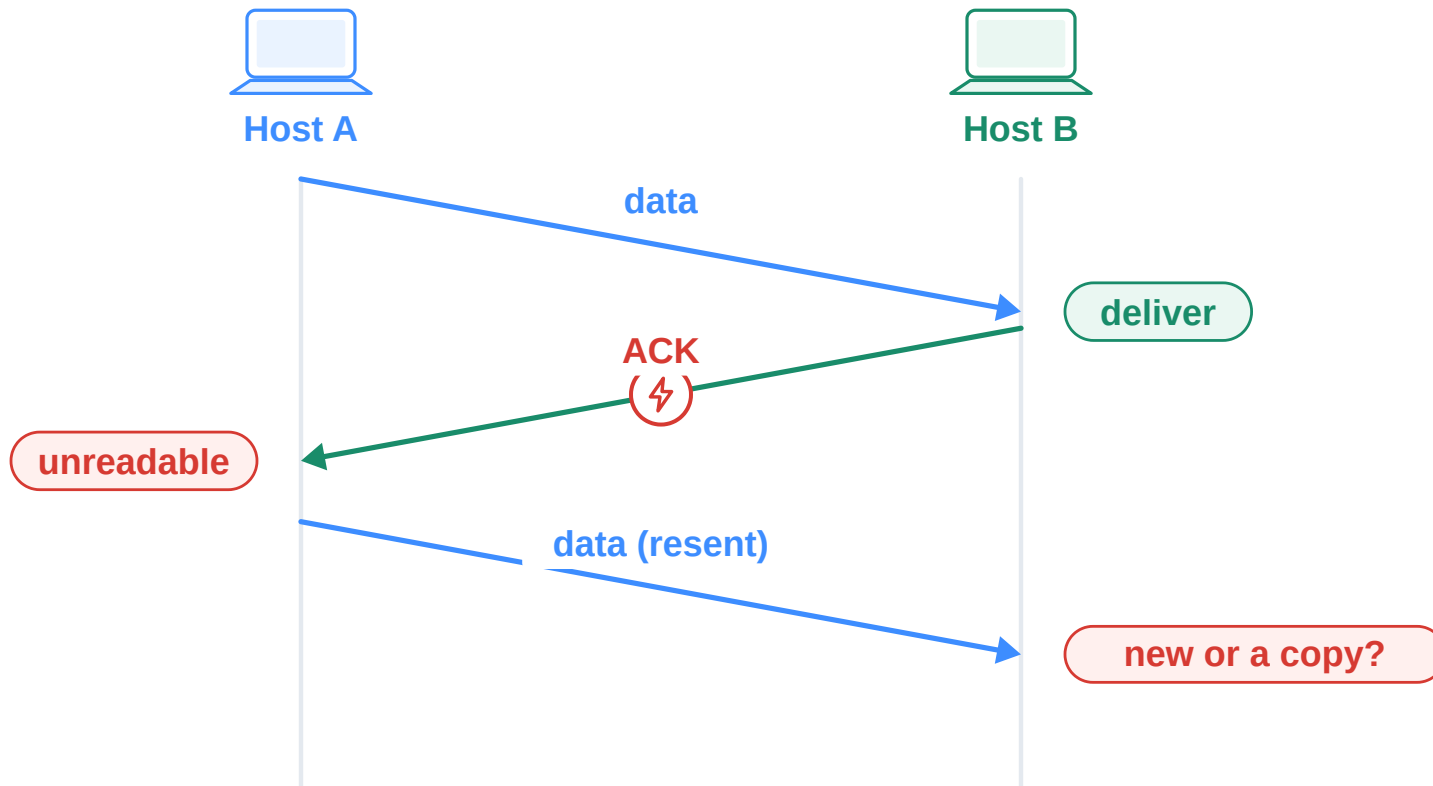
1 Bit errors

2 Loss

3 Delay

4 Reordering

Checkpoint: nothing is lost. Why still a sequence number?



No packet is ever lost.
Why does the protocol still
need a sequence number?



The ACK can be damaged.
A then has to resend, and
without a number B cannot tell
the copy from new data.

Loss is not the reason. Damaged feedback is.

1 Bit errors

2 Loss

3 Delay

4 Reordering

Three questions, three mechanisms: the protocol so far

1 How can B tell that a packet is damaged?

a check field



B recomputes it; a mismatch = damaged

2 How does A learn what arrived at B?

an ACK from B



ACK: send the next; else send it again

3 How does B know that a packet is a repeat?

a 1-bit sequence number



same number as the last one = a repeat

THE DATA PACKET NOW



THE FEEDBACK PACKET



Stop and wait: one packet in flight. A checks the feedback too, and resends the same packet whenever it is not an intact ACK.

⚡ bit flips



🗑 loss

open

⌚ delay

open

🔄 reordering NOT YET

Check, ACK, sequence number: each exists because the protocol before it broke.

1 Bit errors

2 Loss

3 Delay

4 Reordering

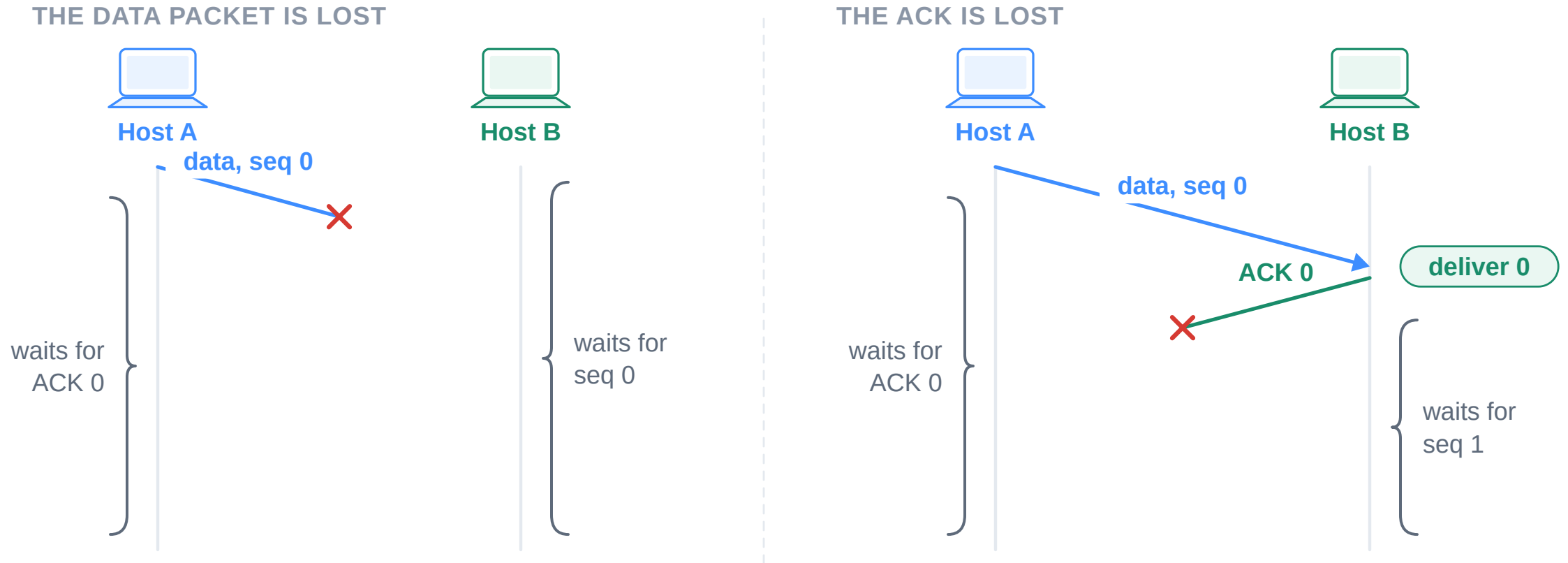
What if the channel loses packets?

02

Loss

A lost packet produces no feedback at all. This part adds the only event the sender can create on its own: a timer.

What happens when a packet is lost?



Nothing in the protocol so far fires without an incoming packet.

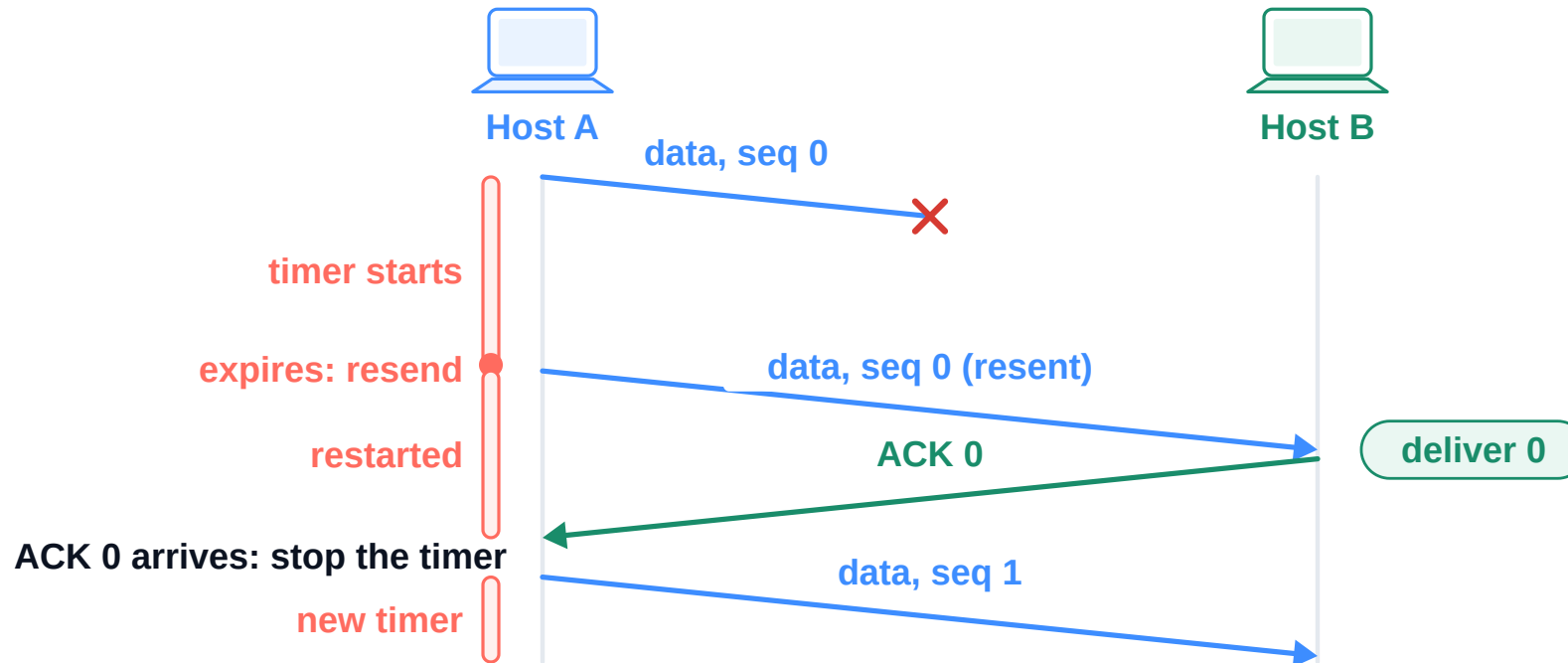
1 Bit errors

2 Loss

3 Delay

4 Reordering

A timer at A turns silence into an event



Rule: start the timer whenever a packet is sent, first time or again; stop it when that packet's ACK arrives.

The timer is the only event the sender can generate on its own.

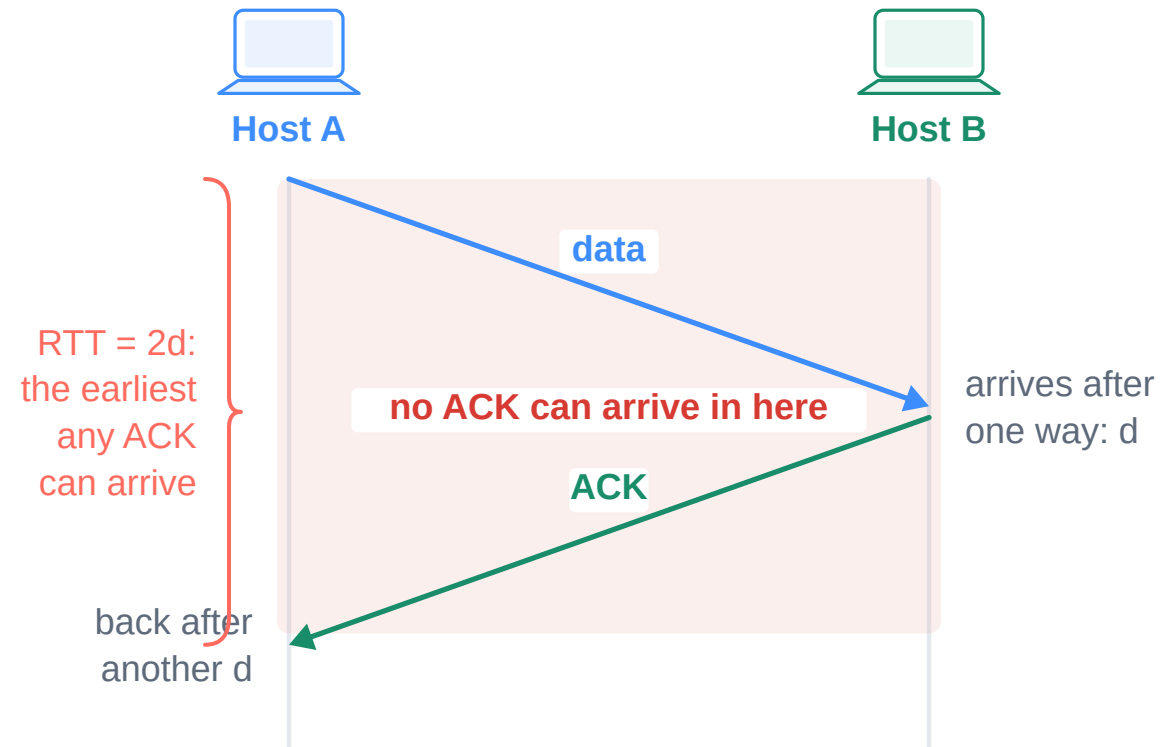
1 Bit errors

2 Loss

3 Delay

4 Reordering

How soon can the ACK possibly arrive?



Information needs time to travel. The earliest ACK is one round trip after the send.

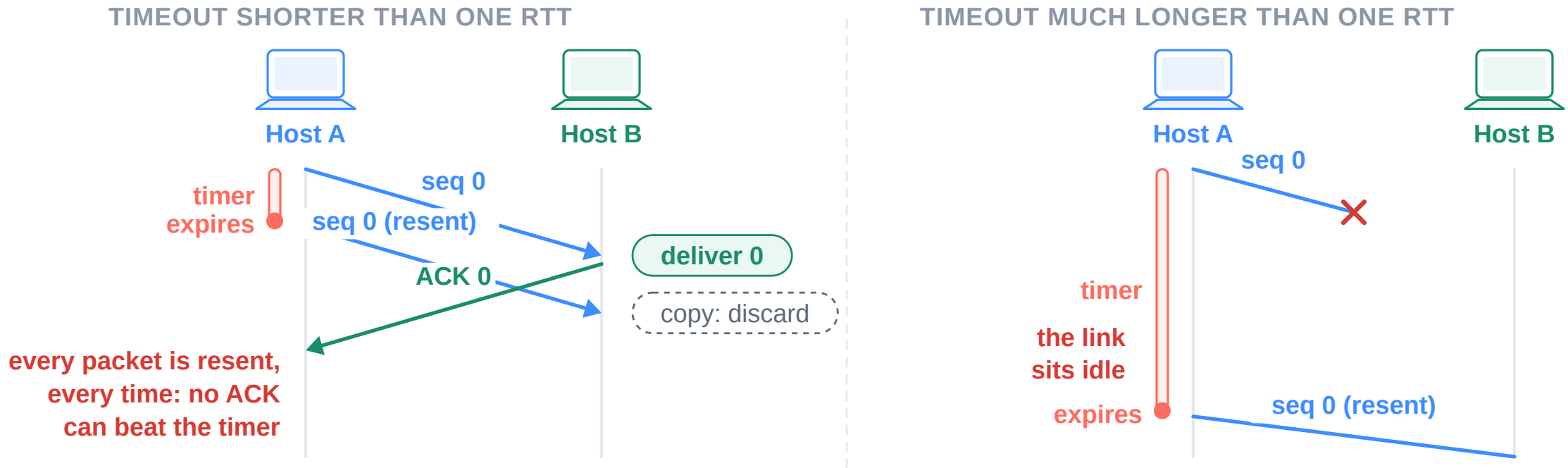
1 Bit errors

2 Loss

3 Delay

4 Reordering

How long should the timer wait?



Shorter than one RTT: the timer always wins, every packet is sent twice. Much longer: the link waits after a real loss.

Longer than one RTT, but not much longer. So A needs to know the round-trip time.

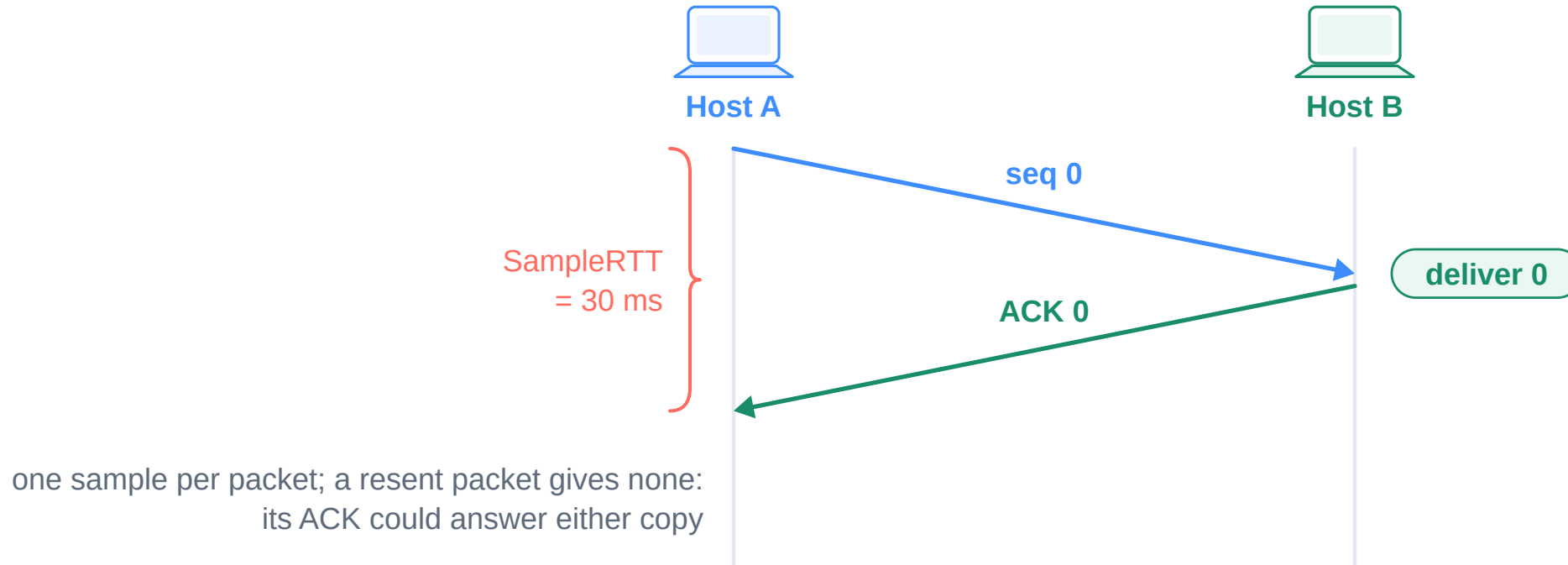
1 Bit errors

2 Loss

3 Delay

4 Reordering

How does A learn the round-trip time?



Measure it: SampleRTT = time from sending a packet to receiving its ACK.

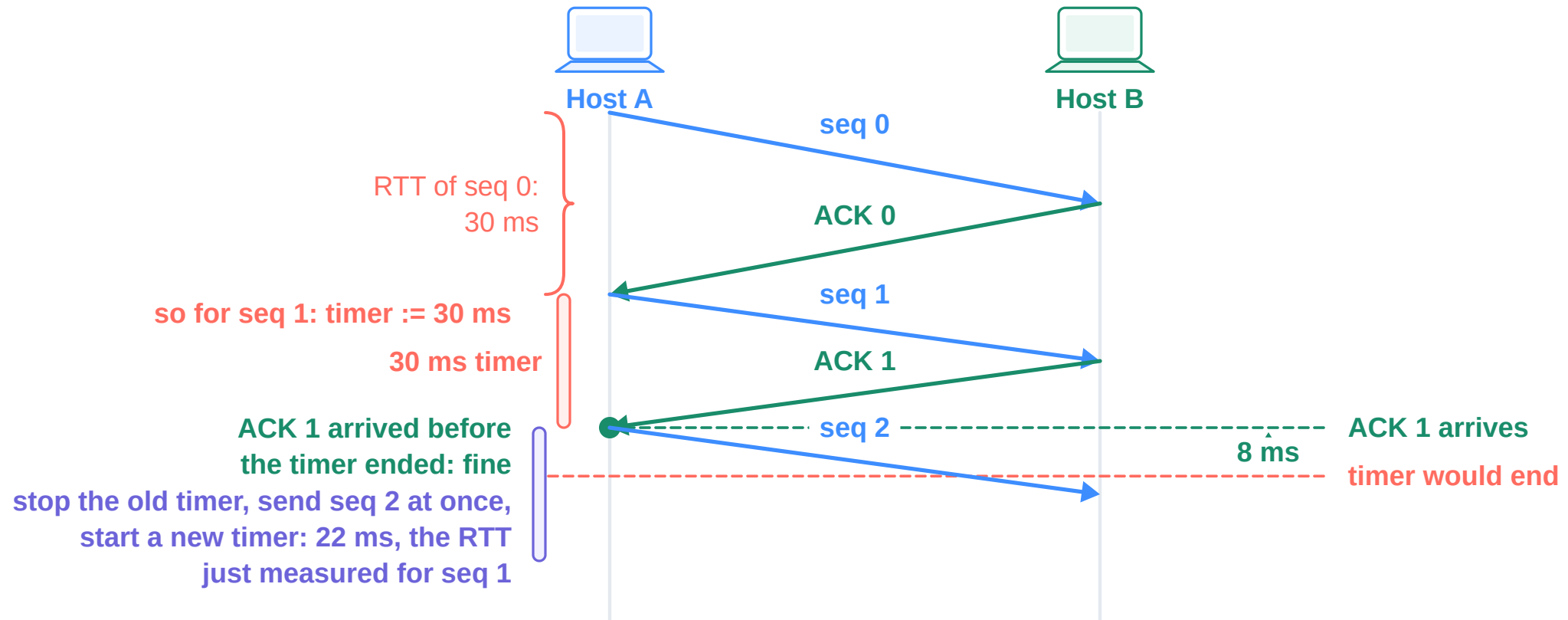
1 Bit errors

2 Loss

3 Delay

4 Reordering

Can A use the last packet's RTT as the timeout?



Here it works: the next round trip was shorter, the ACK beat the 30 ms timer.

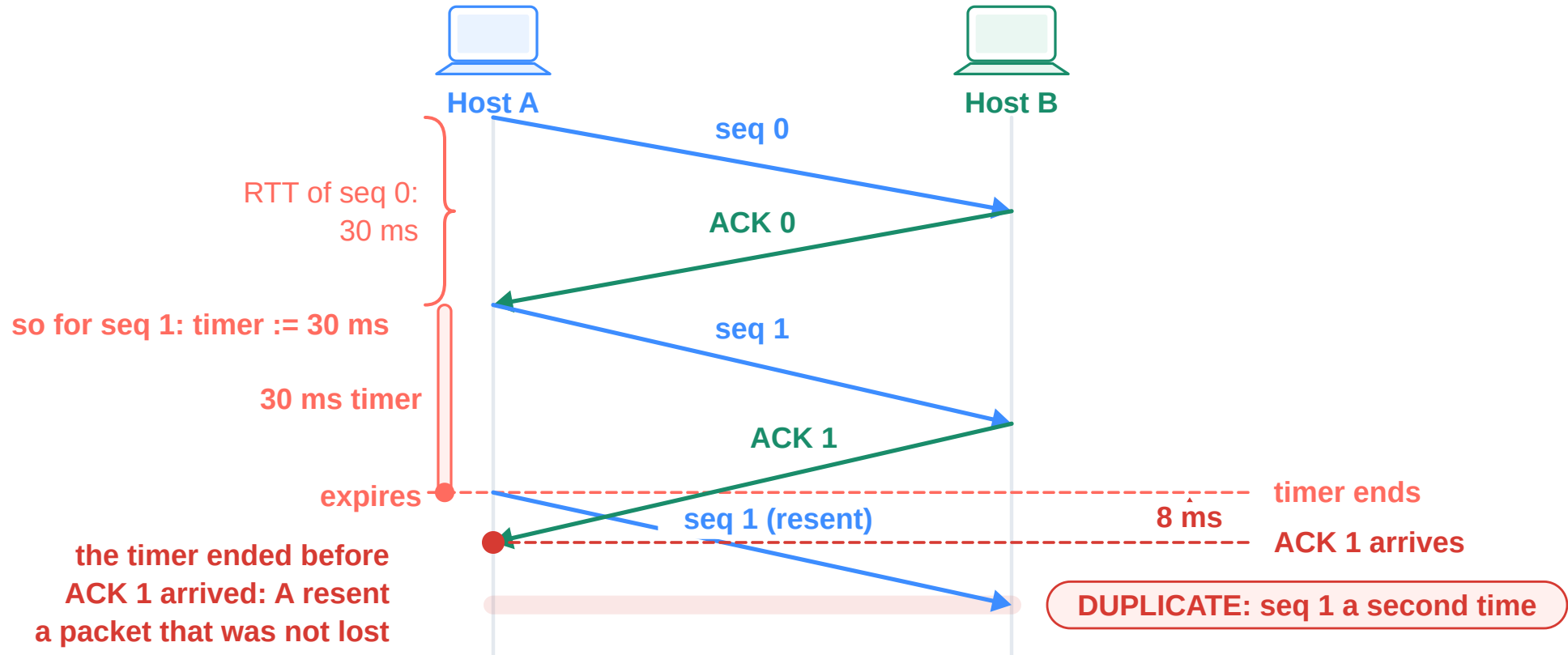
1 Bit errors

2 Loss

3 Delay

4 Reordering

And when the next round trip is longer than the last?



The timer wins, A resends a packet that was not lost. Is the next RTT ever longer? Often.

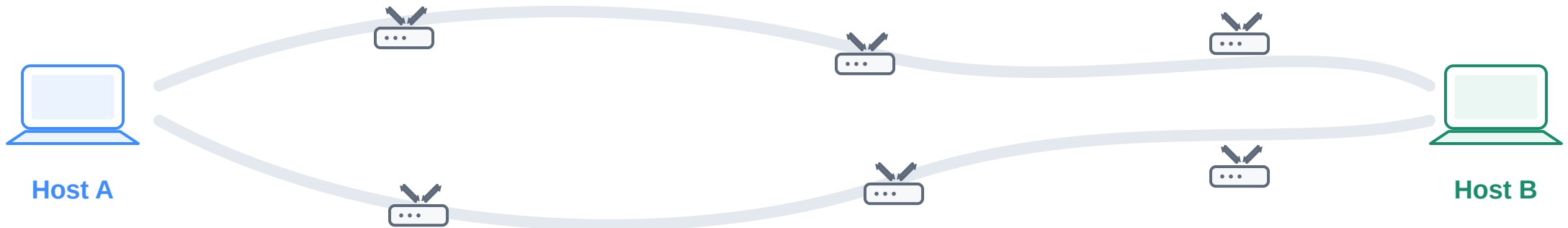
1 Bit errors

2 Loss

3 Delay

4 Reordering

Why is every round trip different?



a different route

each packet may take another path;
a longer path, a longer round trip

a different queue

at each router it waits behind
whatever else is there right now

a busy host

B answers when it gets to it;
A reads the ACK when it gets to it

None of the three is the same for two packets in a row. Every packet has its own round trip.

Route, queues and host load change from packet to packet. So does the RTT.

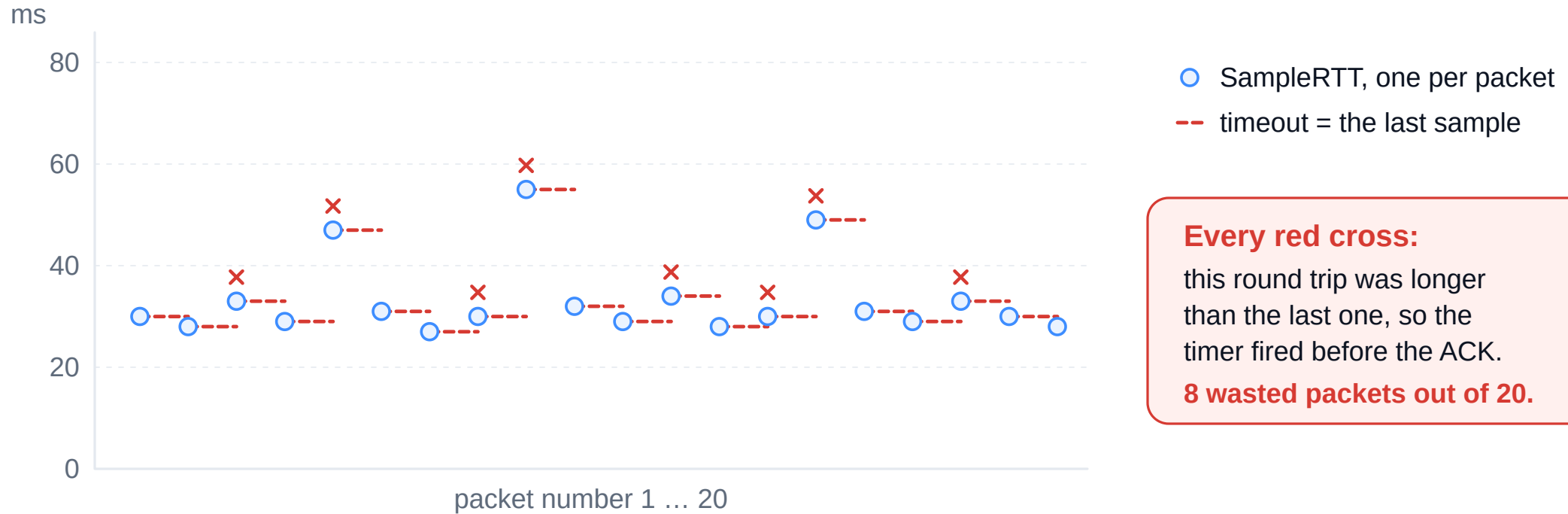
1 Bit errors

2 Loss

3 Delay

4 Reordering

Last RTT as the timeout: every jump up is a false alarm



Fails. Whenever the next RTT is longer than the last, A resends for nothing.

1 Bit errors

2 Loss

3 Delay

4 Reordering

What does A need instead of the last RTT?

1 How to estimate the RTT steadily?

A still needs the RTT. But not one packet's RTT: a single swing must not throw the estimate off.

answer: smooth the samples

2 How much above the estimate?

Round trips scatter around any estimate.
The timeout must sit above it,
or the scatter alone causes resends.

answer: add a margin sized to the scatter

Timeout = a steady estimate of the RTT + a margin for its scatter

Two questions: a steady RTT estimate, and a margin above it. Next, one at a time.

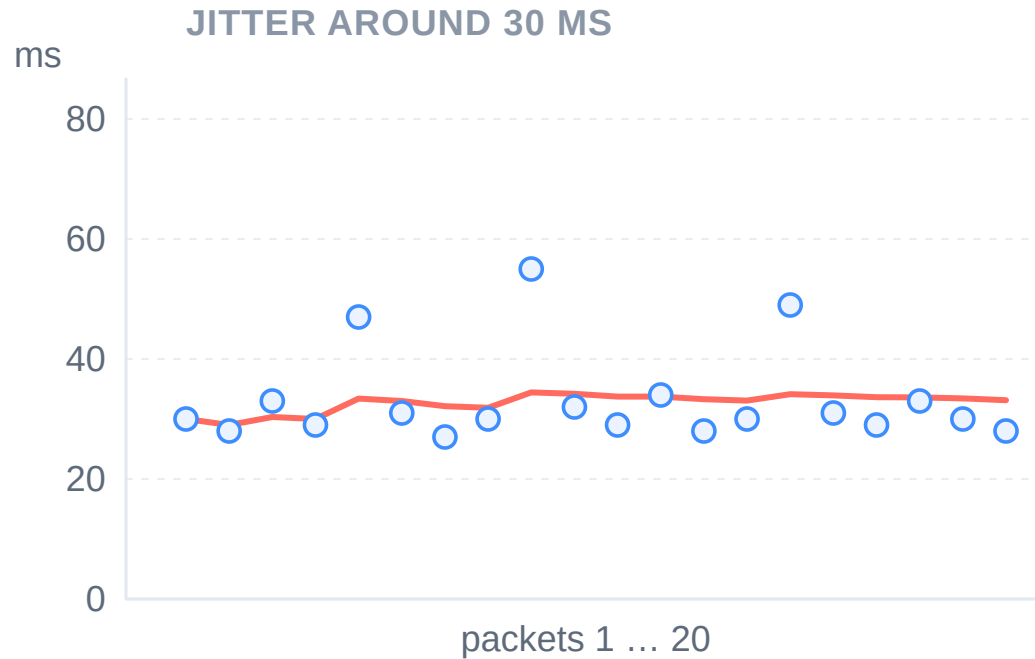
1 Bit errors

2 Loss

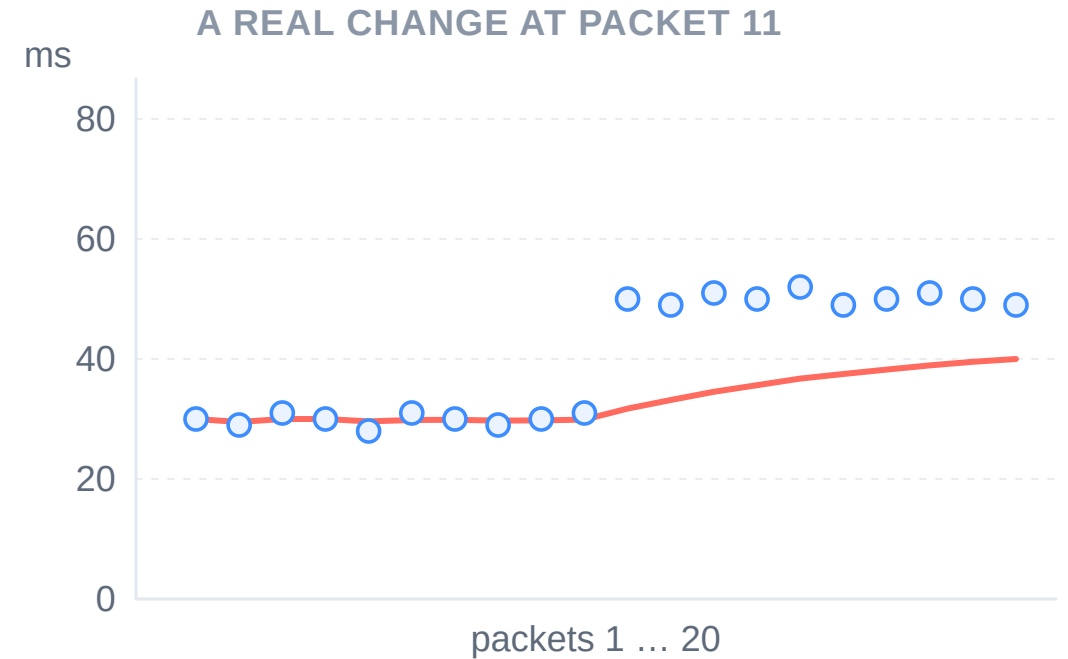
3 Delay

4 Reordering

Simply average all the samples so far?



— average of all samples so far: smooth, yes
but A must store every sample ever measured



✗ sample 1 counts as much as sample 1,000:
after a real change the average never catches up

Smoother, yes. But it keeps every sample and follows a real change too slowly.

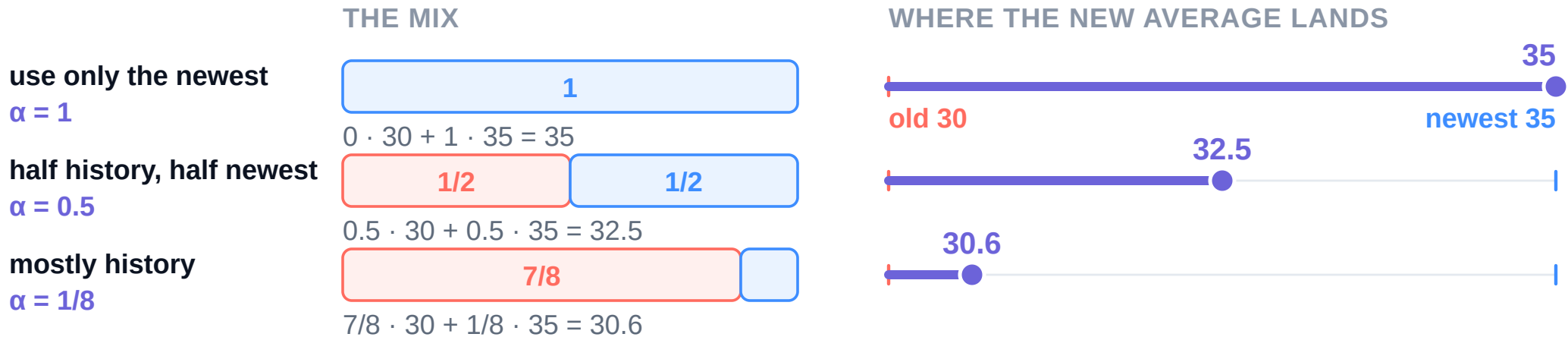
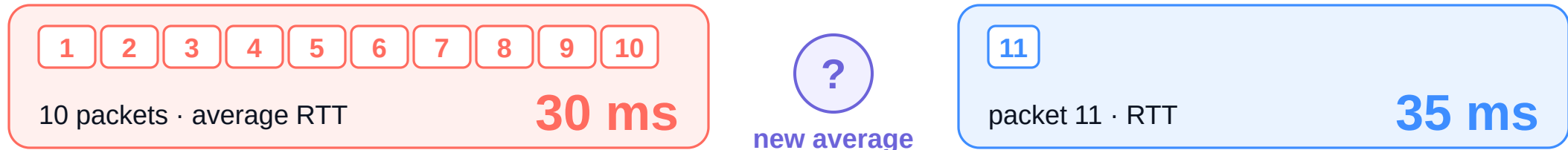
1 Bit errors

2 Loss

3 Delay

4 Reordering

A new RTT sample arrives. How do we update the average?



$$\text{new average} = (1 - \alpha) \cdot \text{old average} + \alpha \cdot \text{newest RTT}$$

EWMA — Exponentially Weighted Moving Average. α is the share of the newest sample; $1 - \alpha$ the share of history.

EWMA: new = $(1 - \alpha) \cdot \text{old} + \alpha \cdot \text{newest}$. α sets the share of the newest sample.

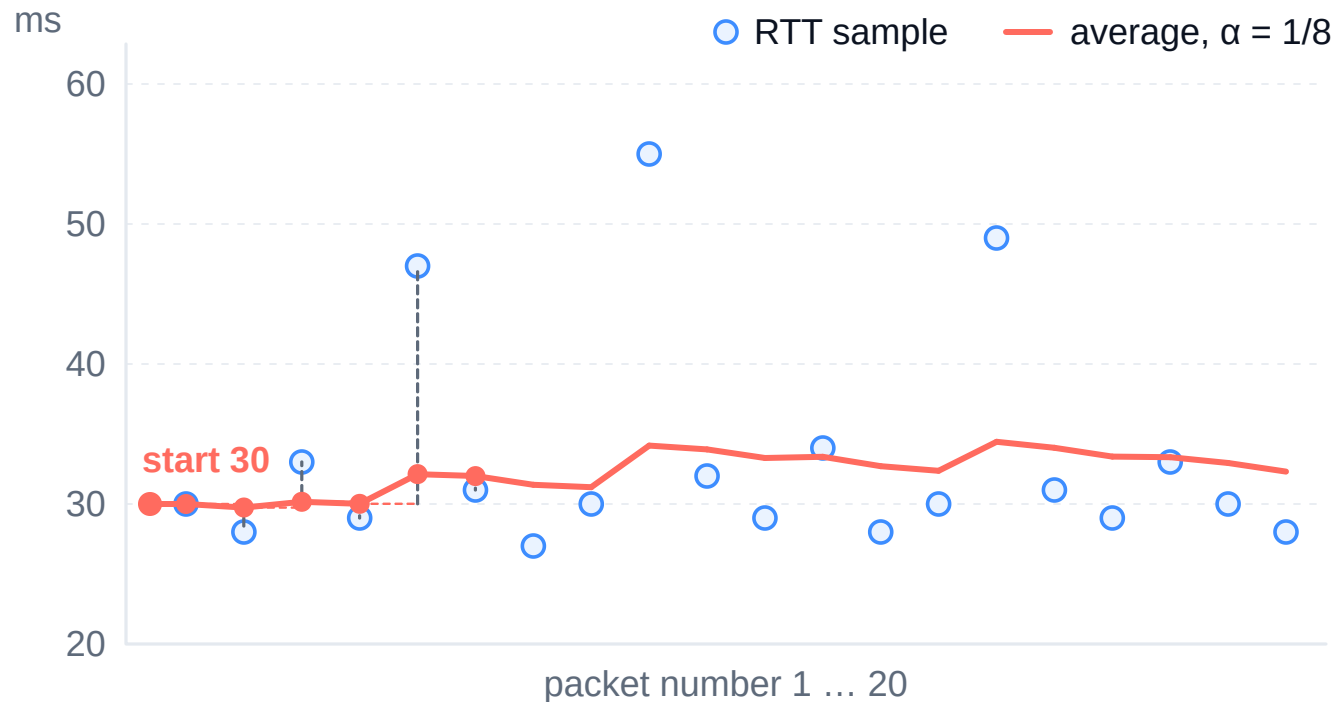
1 Bit errors

2 Loss

3 Delay

4 Reordering

The average, updated packet by packet with $\alpha = 1/8$



packets 7 ... 20

the same rule for every packet:
 $\text{new} = 7/8 \cdot \text{old} + 1/8 \cdot \text{newest}$

The 55 ms spike gets 1/8 of the weight: the average moves 3 ms.

$$\text{new} = (1-\alpha) \cdot \text{old} + \alpha \cdot \text{newest}$$

$\alpha = 1/8$: history 7/8, newest sample 1/8

α sets the share of the newest sample.

Large α : quick to follow, but jumpy.

Small α : calm, but slow after a change.

One stored number. Each new sample gets 1/8 of the weight; history keeps 7/8.

1 Bit errors

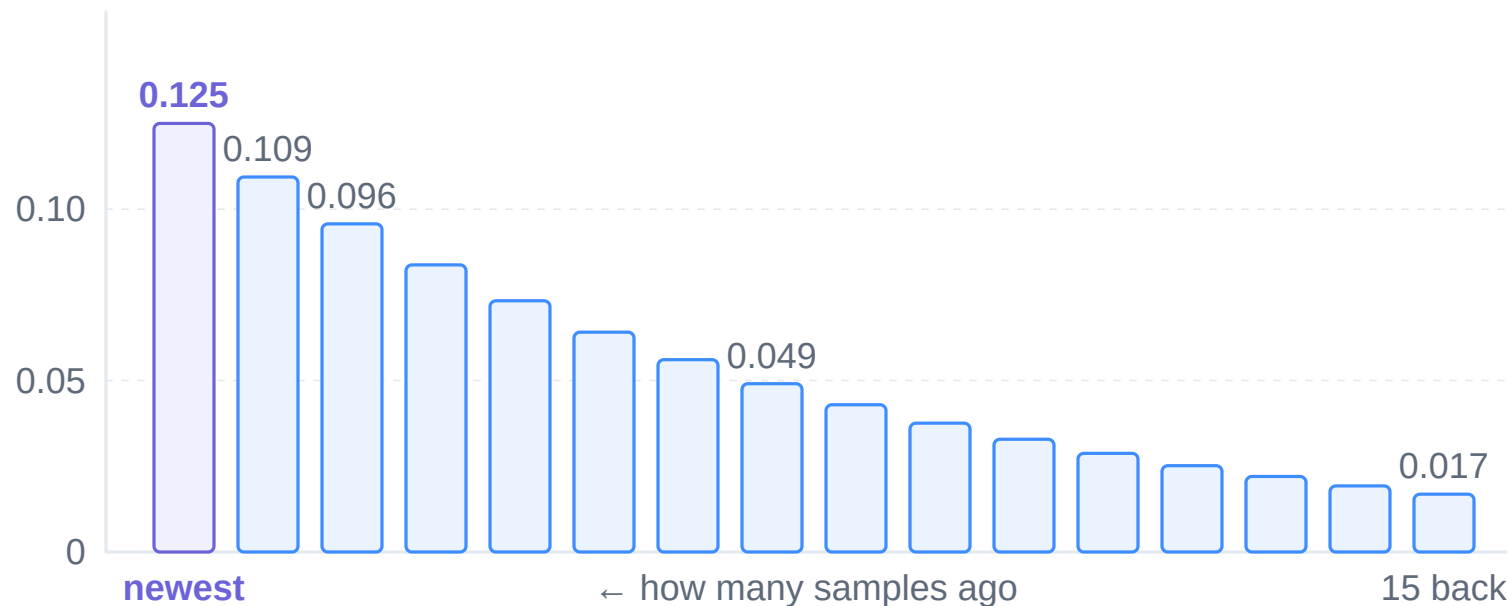
2 Loss

3 Delay

4 Reordering

Why is this called exponential? Old samples fade away

HOW MUCH EACH PAST SAMPLE STILL COUNTS IN TODAY'S ESTIMATE, $\alpha = 1/8$



Unfold the rule once:

the sample before counts $7/8$ as much, the one before that $(7/8)^2$ as much, ...

Every sample is still in there, but a spike 16 packets ago weighs $1/8 \cdot (7/8)^{16} \approx 0.015$.

A weighted average of all past samples, with weights that fade by $7/8$ per step: an exponentially weighted moving average.

Recent samples count most; old ones fade but never vanish. Stored: one number.

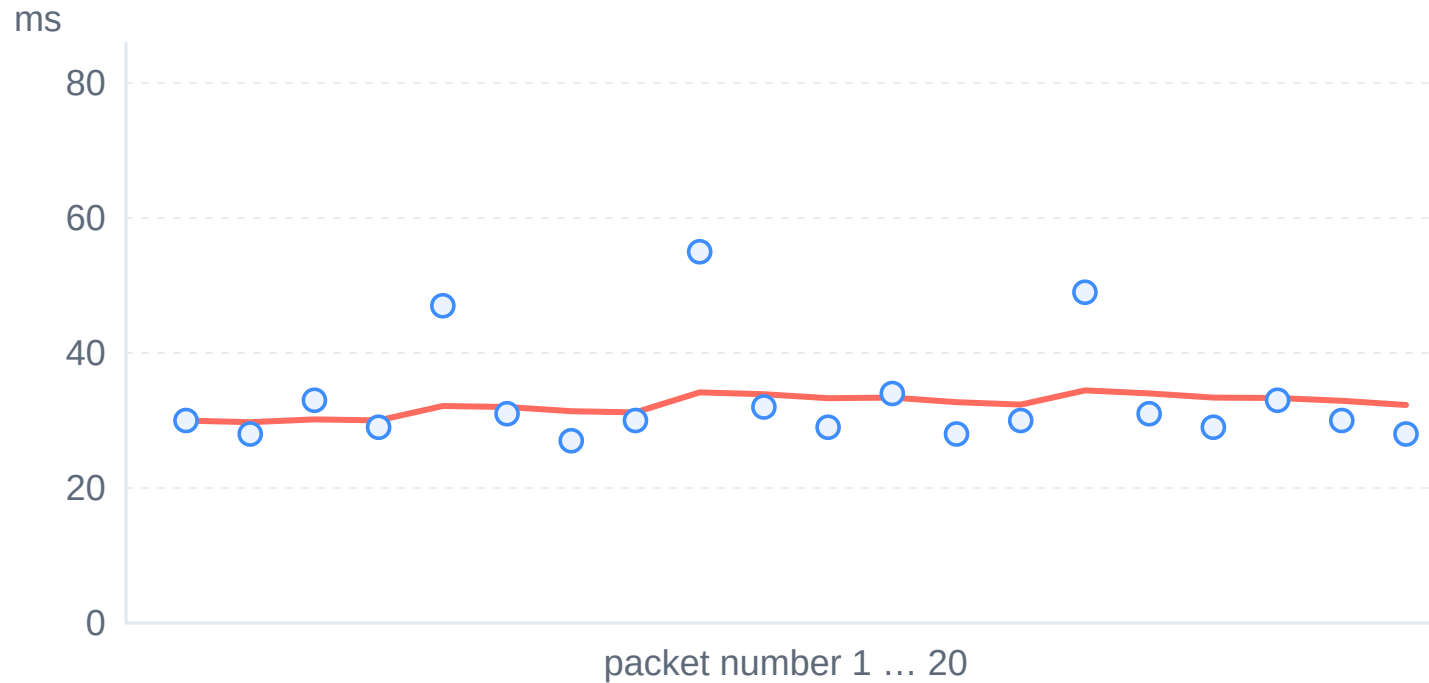
1 Bit errors

2 Loss

3 Delay

4 Reordering

The rule applied to our 20 samples



○ SampleRTT
— EstimatedRTT

$$\text{EstimatedRTT} = (1 - \alpha) \cdot \text{EstimatedRTT} + \alpha \cdot \text{SampleRTT}$$

$\alpha = 1/8 \cdot \text{past vs. newest}$

A weighted balance of two things:
what A believed so far, and what the newest packet just showed. α says how much the newest one counts.

EstimatedRTT follows the samples without jumping with them.

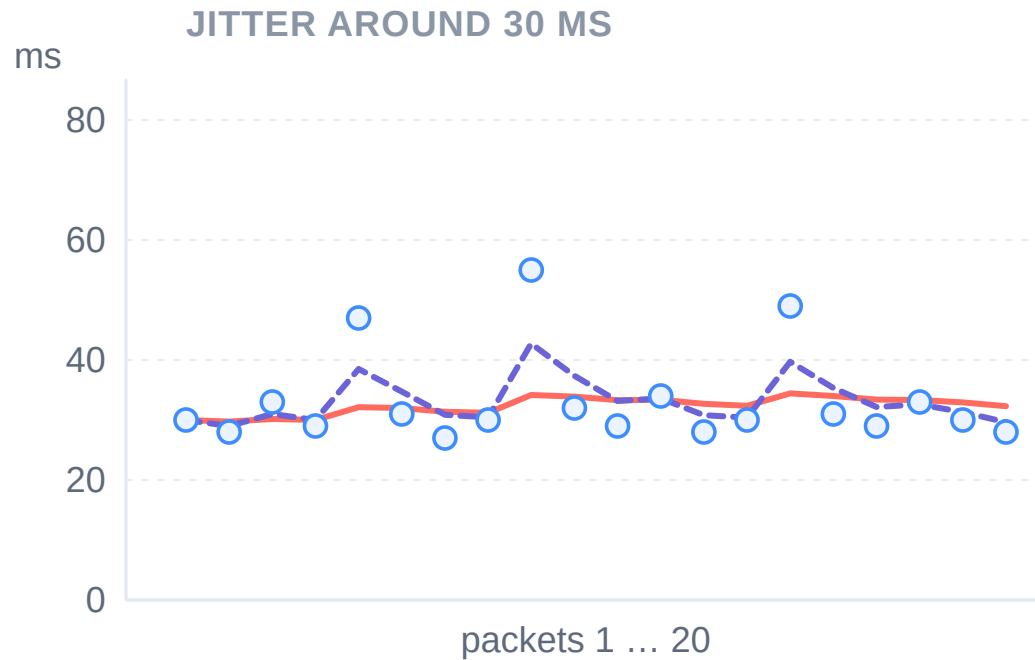
1 Bit errors

2 Loss

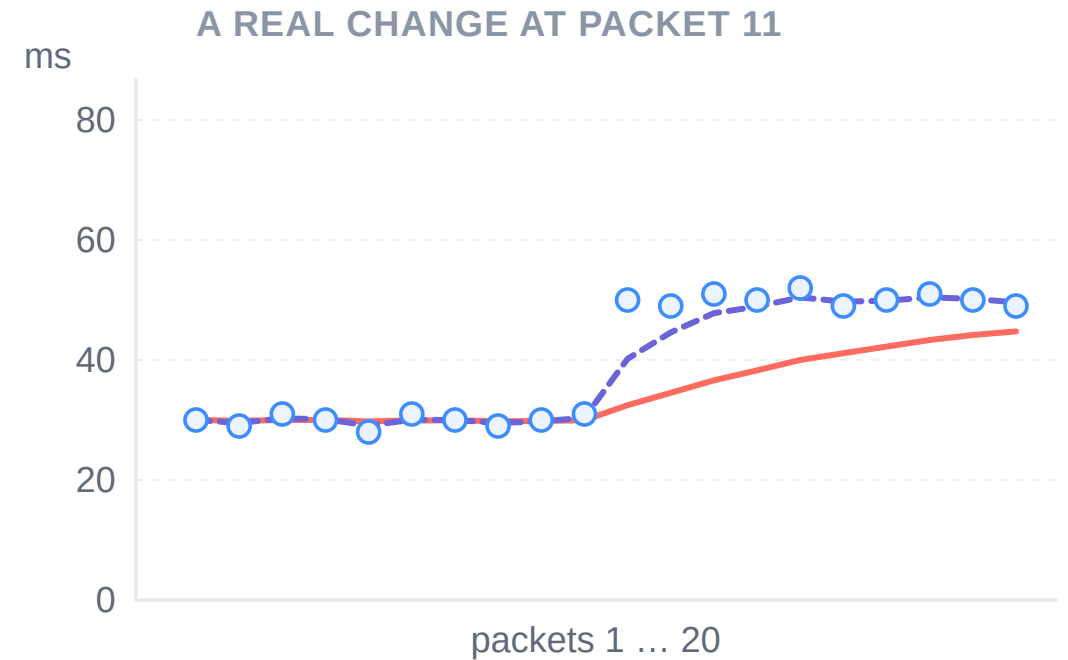
3 Delay

4 Reordering

How much weight should the newest sample get?



— $\alpha = 1/8$: steady; a spike barely moves it,
but a real change takes ~10 packets to follow



--- $\alpha = 1/2$: jumpy; every spike shows,
but a real change is followed in 2 packets

Small α : calm but slow to follow a real change. Large α : quick but jumpy. Practice: 1/8.

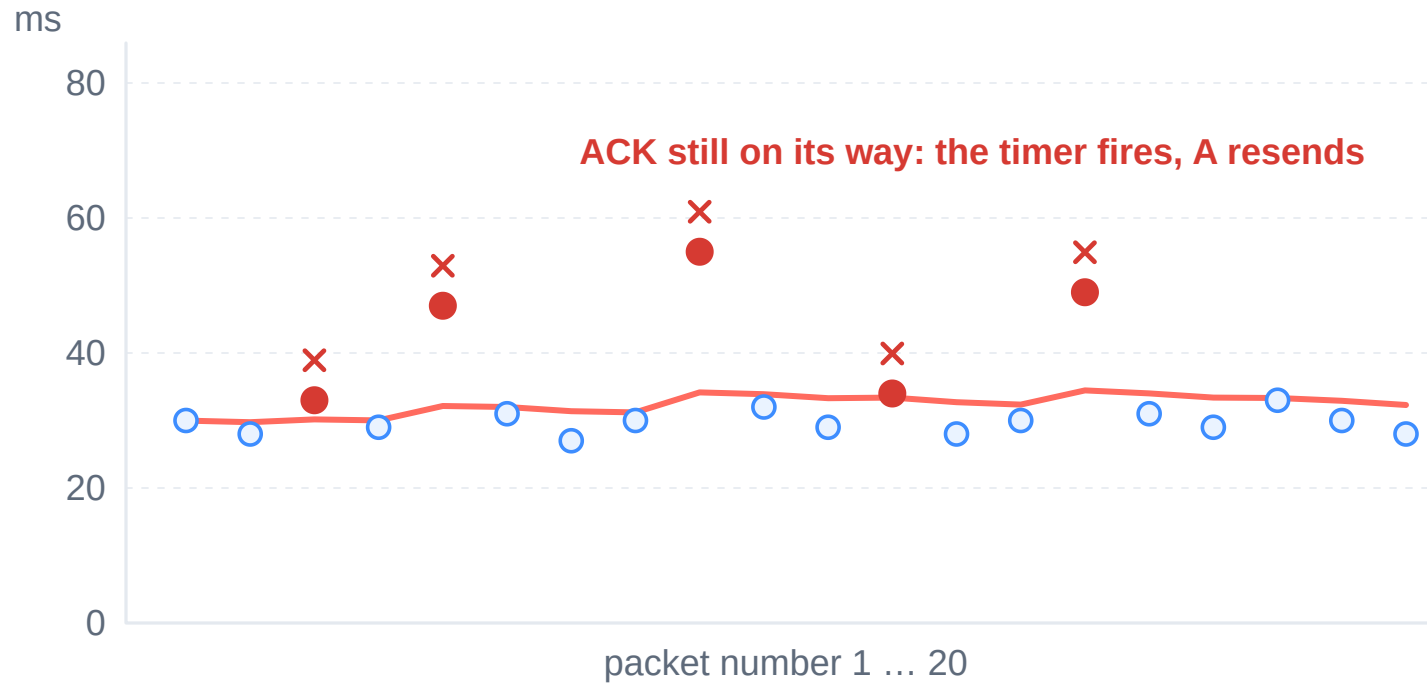
1 Bit errors

2 Loss

3 Delay

4 Reordering

Is EstimatedRTT itself a safe timeout?



○ SampleRTT
— EstimatedRTT

5 samples above the estimate
A timer set to the estimate fires on every one of them:
5 packets resent that A should never have sent.

An average sits in the middle of the samples. The timeout must sit above them: the estimate plus a margin.

No. Every sample above the average is a needless resend: 5 of 20 here.

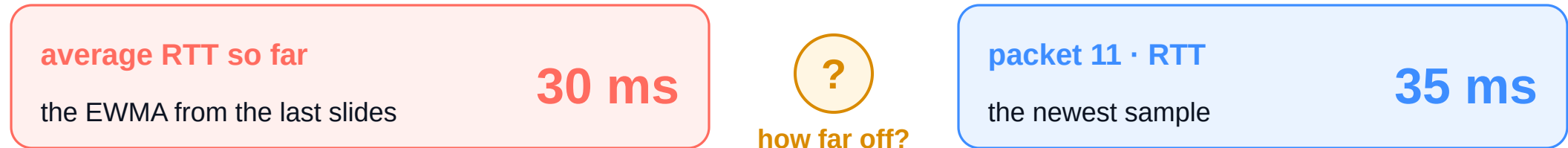
1 Bit errors

2 Loss

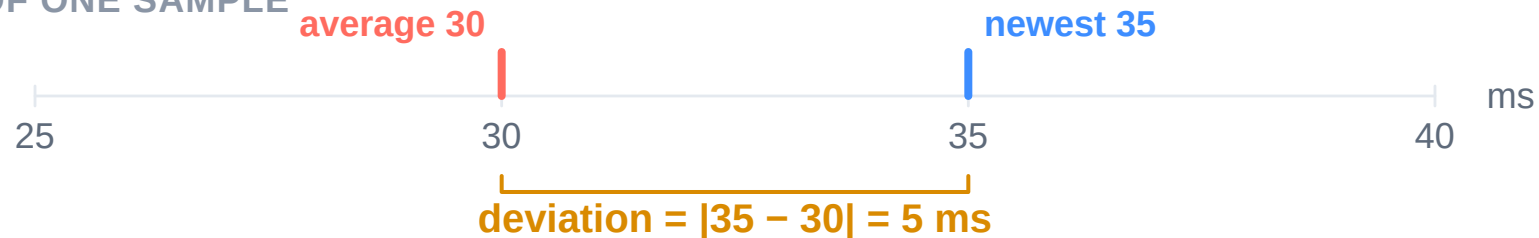
3 Delay

4 Reordering

How far do the samples stray from the average?



THE DEVIATION OF ONE SAMPLE



A TYPICAL DEVIATION: AVERAGE THE DEVIATIONS THE SAME WAY

old DevRTT 4 ms history 3/4 1/4 $\beta = 1/4$ $3/4 \cdot 4 + 1/4 \cdot 5 = 4.25 \text{ ms}$

$$\text{DevRTT} = (1 - \beta) \cdot \text{DevRTT} + \beta \cdot |\text{newest RTT} - \text{average}|$$

the same EWMA rule, applied to the deviations. $\beta = 1/4$ in practice: the newest deviation gets a quarter of the weight.

DevRTT: an EWMA of |sample – average|, the typical distance from the average.

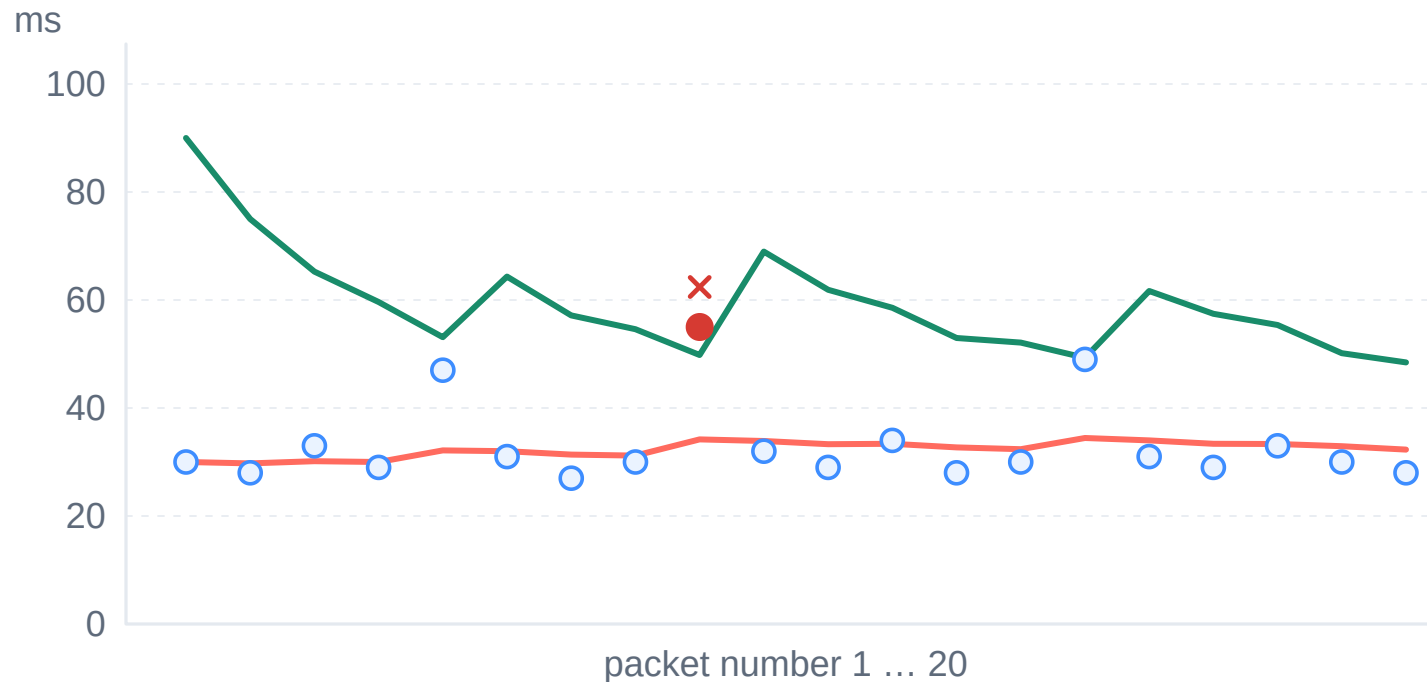
1 Bit errors

2 Loss

3 Delay

4 Reordering

How large should the margin be?



○ SampleRTT
— EstimatedRTT
— Timeout

$$\text{Timeout} = \text{EstimatedRTT} + 4 \cdot \text{DevRTT}$$

Why $4 \cdot \text{DevRTT}$? A sample lands about one DevRTT from the average. Four of them put the timeout above nearly all samples. The margin grows when the samples scatter more, shrinks when calm.

1 early expiry in 20: the 55 ms spike.
The margin follows the past; it cannot foresee a sudden jump. All other jitter: caught.

Timeout = EstimatedRTT + 4 · DevRTT: the margin grows when the round trips vary more.

1 Bit errors

2 Loss

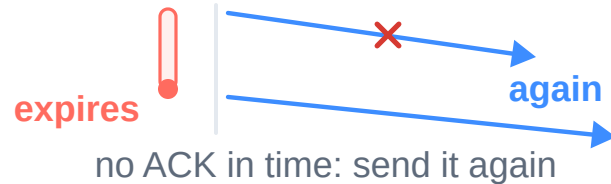
3 Delay

4 Reordering

Two more questions, two more mechanisms

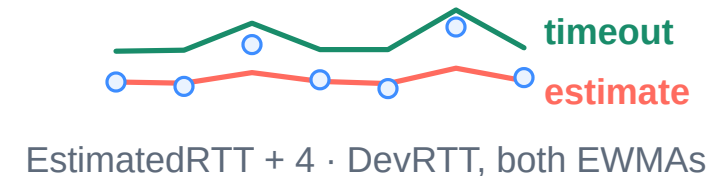
4 What happens when a packet is lost?

a timer at A



5 How long should the timer wait?

a timeout from measured RTTs



WHAT THE SENDER KEEPS NOW



AND BESIDE THE PACKET



Stop and wait, now with a timer: the sender resends on a NAK, on a wrong ACK, or when the timer expires.



A timer, and a timeout learned from the RTTs: the protocol now survives loss.



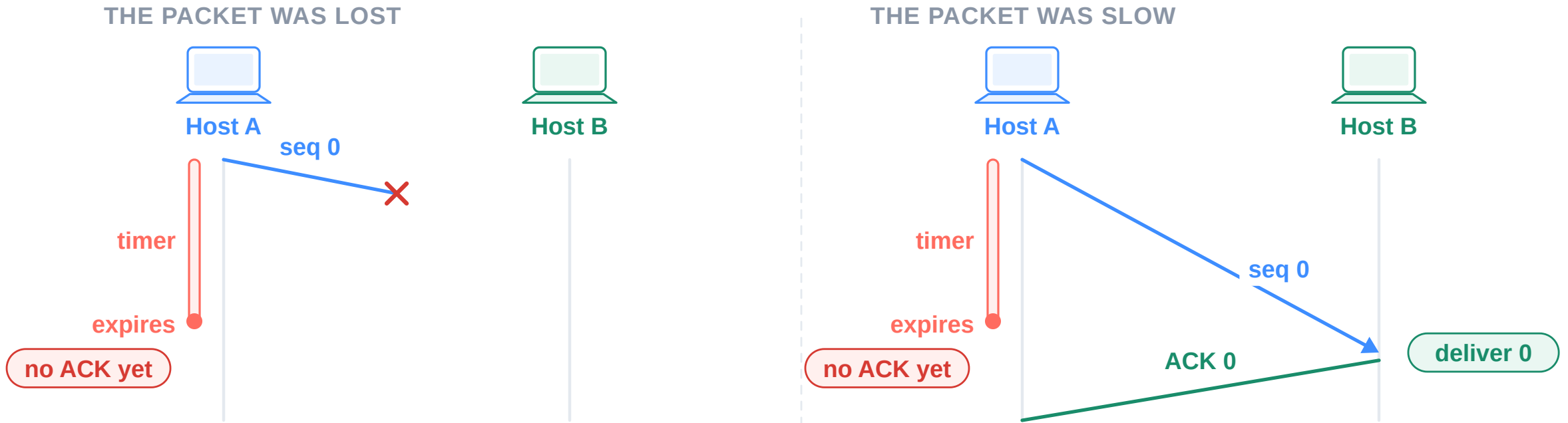
What if the channel only delays packets?

03

Delay

Nothing is lost now, but a packet can take any amount of time. The protocol from Part 2 has a timer: what does it do with a packet that is merely slow?

How does A know that a packet was lost, not slow?



From A's point of view the two pictures are identical until the deadline: no ACK has arrived.

It cannot. A timeout is a guess, and it can be wrong in both directions.

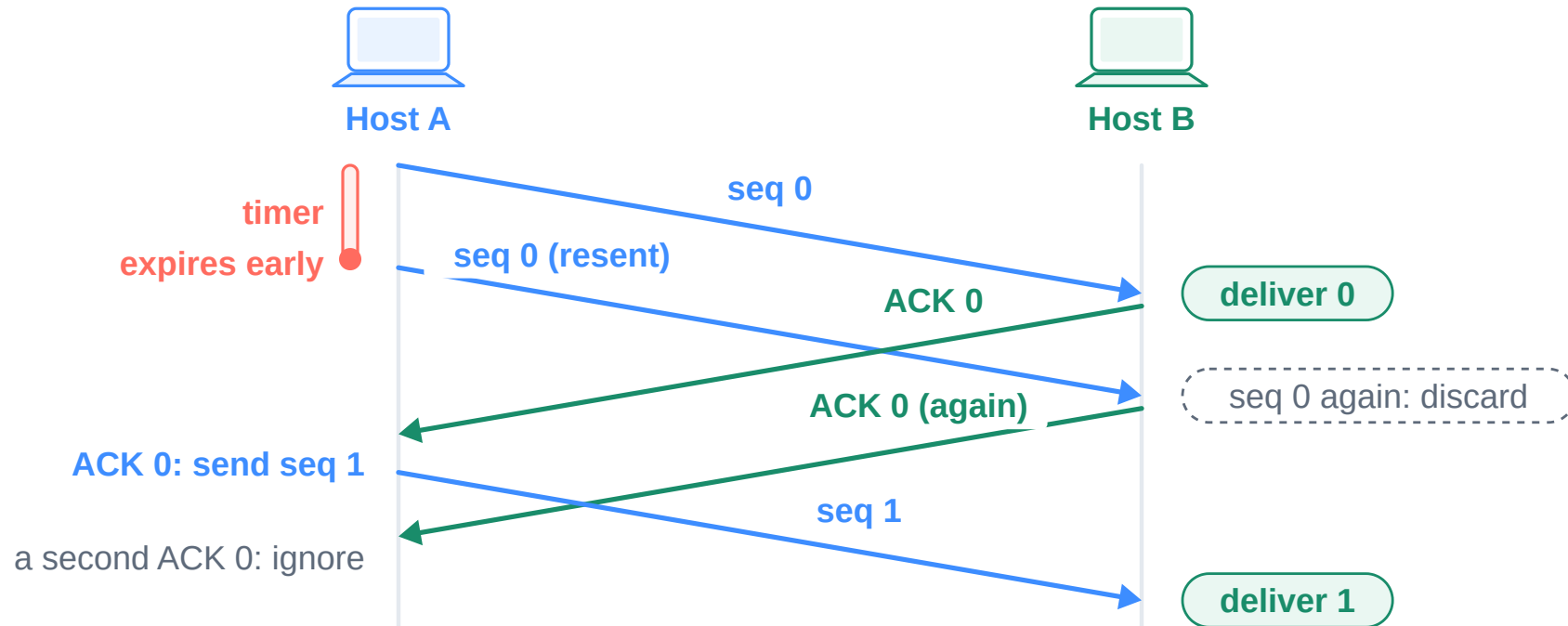
1 Bit errors

2 Loss

3 Delay

4 Reordering

What happens when the timer fires too early?



A duplicate, absorbed by the sequence number: it costs bandwidth, not correctness.

1 Bit errors

2 Loss

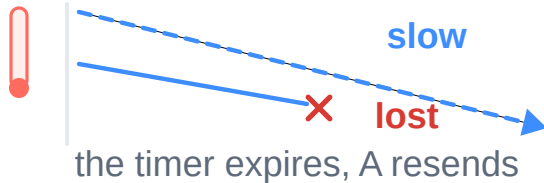
3 Delay

4 Reordering

Delay: two questions, no new mechanism

7 Is a slow packet different from a lost one, for A?

no: treat both as lost



8 What does a needless resend cost?

one duplicate, absorbed

seq 0 (again)

discard, ACK 0

B sees a number it already has and discards the copy

NOTHING NEW IN THE PACKET

seq

data

check

Delay adds no mechanism: the timer and the sequence number from Part 2 already handle it.

⚡ bit flips



🗑 loss



⌚ delay



🔀 reordering

open

A slow packet is handled exactly like a lost one; the sequence number absorbs the copy.

1 Bit errors

2 Loss

3 Delay

4 Reordering

What if packets arrive out of order?

04

Reordering

Two packets can take two routes through the network, and the second one sent can arrive first. With one packet in flight this cannot happen; once several are in flight (next lecture) it can.

Arrivals 1 2 5 6 7 4 3: what does B hand up, and when?

Application

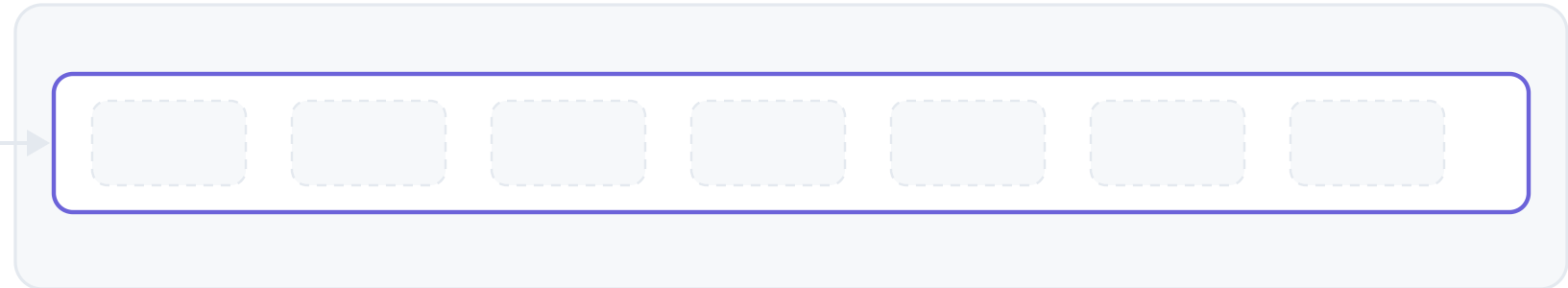
reads the data in order



Transport

receive buffer

FROM THE CHANNEL



3 arrives: hand up 3, then 4, 5, 6, 7 from the buffer.

All seven, in order. The buffer is empty again.

B holds early packets in its buffer and hands them up once the gap before them closes.

1 Bit errors

2 Loss

3 Delay

4 Reordering

Reordering: one question, a buffer

9 What if packets arrive out of order?

a buffer at B



B's buffer: 5, 6, 7 wait for 3 and 4

the sequence number says where each packet belongs; B delivers in that order

NOTHING NEW IN THE PACKET



BESIDE THE RECEIVER



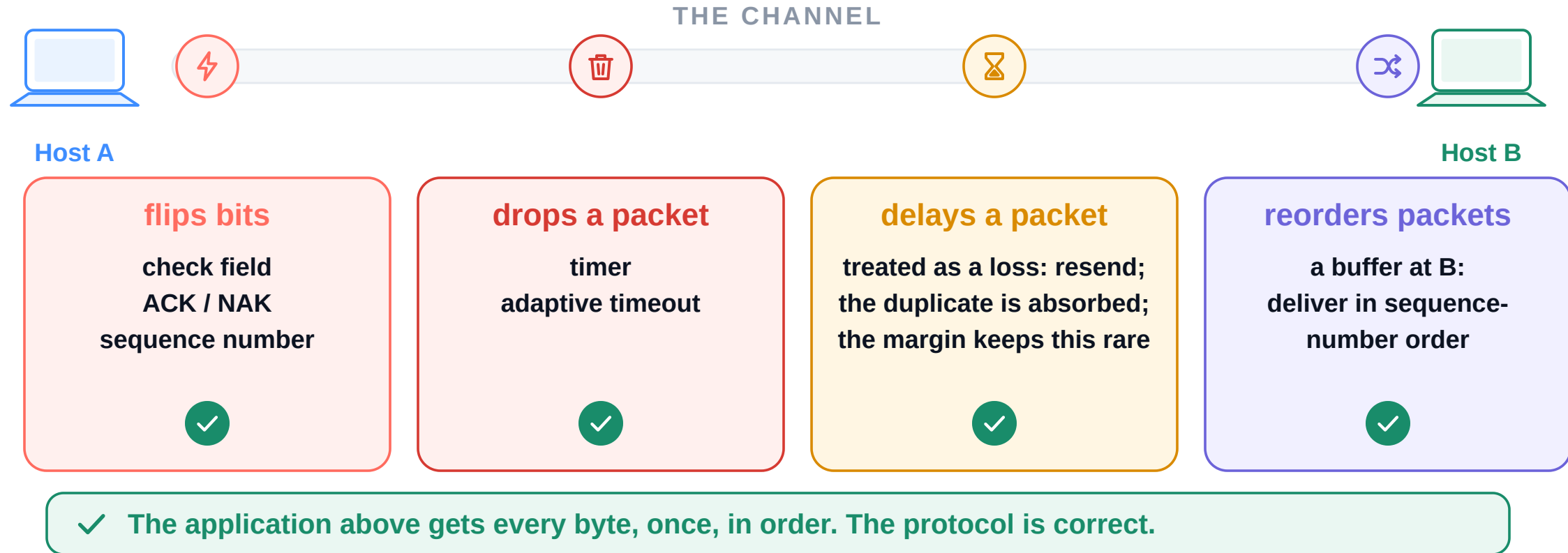
Reordering adds no field: the sequence number from Part 1 already carries the order.



Bit errors, loss, delay, reordering: every channel defect now has an answer.



Where are we? Every channel defect has an answer



Correct, on any channel we have seen. The next question is speed.

1 Bit errors

2 Loss

3 Delay

4 Reordering

Six defects, six mechanisms. Five rows remain

the channel defect	the mechanism it forces
⚡ Bits flip inside a packet	Checksum
☑ A cannot see what arrived at B	ACK from the receiver
# The feedback itself is damaged	Sequence number
⌚ A packet is lost	Timer; resend when it expires
↪ Loss looks the same as delay	Adaptive timeout: $\text{EstimatedRTT} + 4 \cdot \text{DevRTT}$
🔀 Packets arrive out of order	Buffer at B; deliver in sequence order
↻ The link sits idle	—
🔄 One packet in a window is lost	—
🔄 A number is reused too soon	—
📄 Loss is detected late	—
📄 The receiver overflows	—

The five grey rows are the next lectures: several packets in flight, recovery inside a window, then TCP.

Every mechanism so far exists because the protocol before it broke.

1 Bit errors

2 Loss

3 Delay

4 Reordering

Next lecture: correct, but slow. What does waiting cost?



One packet in flight

A sends one packet, then waits a whole round trip for its ACK. The link carries nothing meanwhile.



What does the waiting cost?

1 GB over a 1 Gbps link, RTT 30 ms:
670,000 packets, one round trip each:
about 5.6 hours.



Several packets in flight

Send W packets before the first ACK returns: a window.
How large? What if one is lost?

Pipelining, the sliding window, Go-Back-N and selective repeat: the next lecture.

⚡ bit flips



🗑 loss



⌚ delay



🔄 reordering



Stop-and-wait is correct on any channel. Making it fast is the next problem.

1 Bit errors

2 Loss

3 Delay

4 Reordering